第 3 章 栈、队列和数组

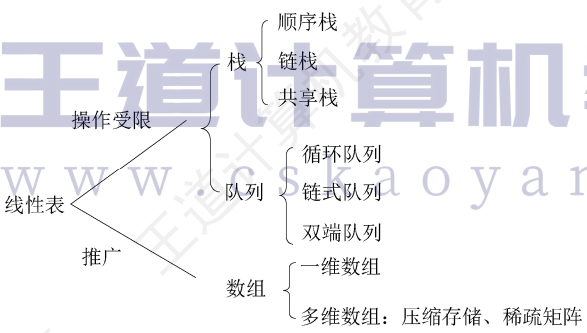
【考纲内容】

（一）栈和队列的基本概念
（二）栈和队列的顺序存储结构
（三）栈和队列的链式存储结构
（四）多维数组的存储
（五）特殊矩阵的压缩存储
（六）栈、队列和数组的应用

扫一扫

视频讲解

【知识框架】



【复习提示】

本章通常以选择题的形式考查，题目不算难，但命题的形式比较灵活，其中栈（出入栈的过程、出栈序列的合法性）和队列的操作及其特征是重点。因为它们均是线性表的应用和推广，所以也容易出现在算法设计题中。此外，栈和队列的顺序存储、链式存储及其特点，双端队列的特点，栈和队列的常见应用，以及数组和特殊矩阵的压缩存储都是读者必须掌握的内容。

3.1 栈

3.1.1 栈的基本概念

1．栈的定义

命题追踪 ▶▶ 栈的特点（2017）

栈（Stack）是只允许在一端进行插入或删除操作的线性表。首先栈是一种线性表，但限定这种线性表只能在某一端进行插入和删除操作，如图 3.1 所示。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

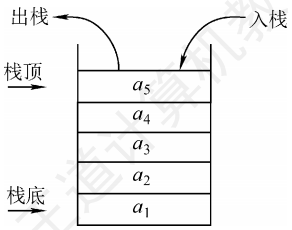


图 3.1　栈的示意图

栈顶（Top）：线性表允许进行插入和删除操作的那一端。

栈底（Bottom）：固定的，不允许进行插入和删除操作的另一端。

空栈：不含任何元素的空表。

命题追踪 ▶▶ 入栈序列和出栈序列之间的关系（2022）

命题追踪 ▶▶ 特定条件下的出栈序列分析（2010、2011、2013、2018、2020）

假设某个栈 $S=(a_1, a_2, a_3, a_4, a_5)$，如图 3.1 所示，则 a_1 为栈底元素，a_5 为栈顶元素。栈只能在栈顶进行插入和删除操作，入栈次序依次为 a_1, a_2, a_3, a_4, a_5，而出栈次序为 a_5, a_4, a_3, a_2, a_1。由此可见，栈的操作特性可以明显地概括为后进先出（Last In First Out，LIFO）。

注　意

每接触一种新的数据结构，都应从其逻辑结构、存储结构和运算三个方面着手。

2．栈的基本操作

各种辅导书中给出的基本操作的名称不尽相同，但所表达的意思大致是一样的。这里我们以严蔚敏编写的教材为准给出栈的基本操作，希望读者能熟记下面的基本操作。

- `InitStack(&S)`：初始化一个空栈 S。
- `StackEmpty(S)`：判断一个栈是否为空，若栈 S 为空则返回 true，否则返回 false。
- `Push(&S,x)`：入栈，若栈 S 未满，则将 x 加入使之成为新栈顶。
- `Pop(&S,&x)`：出栈，若栈 S 非空，则弹出栈顶元素，并用 x 返回。
- `GetTop(S,&x)`：读栈顶元素，但不出栈，若栈 S 非空，则用 x 返回栈顶元素。
- `DestroyStack(&S)`：销毁栈，并释放栈 S 占用的存储空间（“&”表示引用调用）。

在解答算法题时，若题干未做出限制，则也可直接使用这些基本的操作函数。

栈的数学性质：当 n 个不同元素入栈时，出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^{n}$。这个公式称为卡特兰数（Catalan）公式，可采用数学归纳法证明，有兴趣的读者可以参考组合数学教材。

3.1.2　栈的顺序存储结构

栈是一种操作受限的线性表，类似于线性表，它也有对应的两种存储方式。

1．顺序栈的实现

采用顺序存储的栈称为顺序栈，它利用一组地址连续的存储单元存放自栈底到栈顶的数据元素，同时附设一个指针（top）指示当前栈顶元素的位置。

栈的顺序存储类型可描述为

```
#define MaxSize 50              //定义栈中元素的最大个数
typedef struct{
```

配套课程公众号[考机研]
永久联系微信:4550060

```
    Elemtype data[MaxSize];      //存放栈中元素
    int top;                     //栈顶指针
}SqStack;
```

栈顶指针：S.top，初始时设置S.top=-1；栈顶元素：S.data[S.top]。

入栈操作：栈不满时，栈顶指针先加1，再送值到栈顶。

出栈操作：栈非空时，先取栈顶元素，再将栈顶指针减1。

栈空条件：S.top==-1；栈满条件：S.top==MaxSize-1；栈长：S.top+1。

另一种常见的方式是：初始设置栈顶指针S.top=0；入栈时先将值送到栈顶，栈顶指针再加1；出栈时，栈顶指针先减1，再取栈顶元素；栈空条件是S.top==0；栈满条件是S.top==MaxSize。

顺序栈的入栈操作受数组上界的约束，当对栈的最大使用空间估计不足时，有可能发生栈上溢，此时应及时向用户报告消息，以便及时处理，避免出错。

注 意

栈和队列的判空、判满条件，会因实际给出的条件不同而变化，下面的代码实现是在栈顶指针初始化为-1的条件下的相应方法，而其他情况则需具体问题具体分析。

2．顺序栈的基本操作

命题追踪 ▶▶ 出/入栈操作的模拟（2009）

栈操作的示意图如图3.2所示，图3.2(a)是空栈，图3.2(c)是A、B、C、D、E共5个元素依次入栈后的结果，图3.2(d)是在图3.2(c)之后E、D、C的相继出栈，此时栈中还有2个元素，或许最近出栈的元素C、D、E仍在原先的单元存储着，但top指针已经指向了新的栈顶，元素C、D、E已不在栈中，读者应通过该示意图深刻理解栈顶指针的作用。

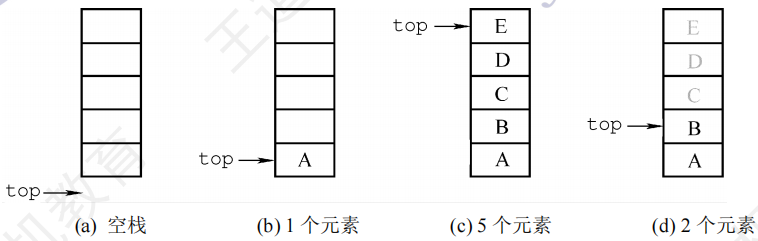


图3.2 栈顶指针和栈中元素之间的关系

下面是顺序栈上常用的基本操作的实现。

（1）初始化

```
void InitStack(SqStack &S){
    S.top=-1;                    //初始化栈顶指针
}
```

（2）判栈空

```
bool StackEmpty(SqStack S){
    if(S.top==-1)                //栈空
        return  true;
    else                         //不空
        return  false;
}
```

（3）入栈

```
bool Push(SqStack &S,ElemType x){
```

配套课程公众号[考机研]
永久联系微信:4550060

```
    if(S.top==MaxSize-1)          //栈满，报错
        return  false;
    S.data[++S.top]=x;            //指针先加 1，再入栈
    return  true;
}
```

当栈不满时，top 先加 1，再入栈。若初始时将 top 定义为 0，函数 3 和 4 应如何改写？

（4）出栈

```
bool Pop(SqStack &S,ElemType &x){
    if(S.top==-1)                 //栈空，报错
        return  false;
    x=S.data[S.top--];            //先出栈，指针再减 1
    return  true;
}
```

（5）读栈顶元素

```
bool GetTop(SqStack S,ElemType &x){
    if(S.top==-1)                 //栈空，报错
        return  false;
    x=S.data[S.top];              //x 记录栈顶元素
    return  true;
}
```

仅为读取栈顶元素，并没有出栈操作，因此原栈顶元素依然保留在栈中。

注　意

这里的 top 指的是栈顶元素。于是，入栈操作为 S.data[++S.top]=x，出栈操作为 x=S.data[S.top--]。若栈顶指针初始化为 S.top=0，即 top 指向栈顶元素的下一位置，则入栈操作变为 S.data[S.top++]=x；出栈操作变为 x=S.data[--S.top]。相应的栈空、栈满条件也会发生变化。请读者仔细体会其中的不同之处，做题时要灵活应变。

王道计算机教育 www.cskaoyan.com

3．共享栈

利用栈底位置相对不变的特性，可让两个顺序栈共享一个一维数组空间，将两个栈的栈底分别设置在共享空间的两端，两个栈顶向共享空间的中间延伸，如图 3.3 所示。

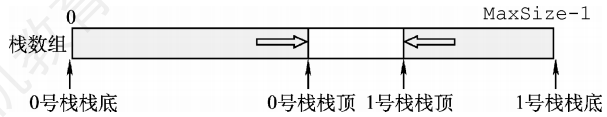


图 3.3　两个顺序栈共享存储空间

两个栈的栈顶指针都指向栈顶元素，top0=-1 时 0 号栈为空，top1=MaxSize 时 1 号栈为空；仅当两个栈顶指针相邻（top1-top0=1）时，判断为栈满。当 0 号栈入栈时 top0 先加 1 再赋值，1 号栈入栈时 top1 先减 1 再赋值；出栈时则刚好相反。

共享栈是为了更有效地利用存储空间，两个栈的空间相互调节，只有在整个存储空间被占满时才发生上溢。其存取数据的时间复杂度均为 $O(1)$，所以对存取效率没有什么影响。

3.1.3　栈的链式存储结构

采用链式存储的栈称为链栈，链栈的优点是便于多个栈共享存储空间和提高其效率，且不存在栈满上溢的情况。通常采用单链表实现链栈，并规定所有操作都是在单链表的表头进行的。这里规定链栈没有头结点，Lhead 指向栈顶元素，如图 3.4 所示。

配套课程公众号[考机研]
永久联系微信:4550060

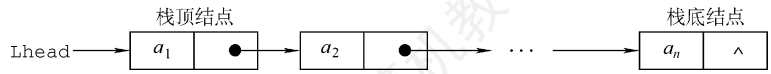


图 3.4 栈的链式存储

栈的链式存储类型可描述为

```
typedef struct Linknode{
    ElemType data;                  //数据域
    struct Linknode *next;          //指针域
}LiStack;                           //栈类型定义
```

采用链式存储，便于结点的插入与删除。链栈的操作与链表类似，入栈和出栈的操作都在链表的表头进行。需要注意的是，对于带头结点和不带头结点的链栈，具体的实现会有所不同。

3.1.4 本节试题精选

一、单项选择题

01. 栈和队列具有相同的（ ）。

A. 抽象数据类型 B. 逻辑结构 C. 存储结构 D. 运算

02. 栈是一种（ ）。

A. 顺序存储的线性结构 B. 链式存储的非线性结构

C. 限制存取点的线性结构 D. 限制存取点的非线性结构

03. 下列选项中，（ ）不是栈的基本操作。

A. 删除栈顶元素 B. 删除栈底元素

C. 判断栈是否为空 D. 将栈置为空栈

04. 假定用数组 a[n]存储一个栈，初始栈顶指针 top=-1，则元素 x 入栈的操作是（ ）。

A. a[--top]=x B. a[top--]=x C. a[++top]=x D. a[top++]=x

05. 假定用数组 a[1…n]存储一个栈，初始栈顶指针 top = 1，则元素 x 入栈的操作是（ ）。

A. data[top--]=x B. data[top++]=x

C. data[--top]=x D. data[++top]=x

06. 假定用数组 a[1…n]存储一个栈，初始栈顶指针 top=n+1，则元素 x 入栈的操作是（ ）。

A. data[--top]=x B. data[top++]=x

C. data[top--]=x D. data[++top]=x

07. 设有一个空栈，栈顶指针为 1000H，栈向高地址方向增长，每个元素占一个存储单元，执行 push、push、pop、push、pop、push、pop、push 操作后，栈顶指针为（ ）。

A. 1002H B. 1003H C. 1004H D. 1005H

08. 和顺序栈相比，链栈有一个比较明显的优势，即（ ）。

A. 通常不会出现栈满的情况 B. 通常不会出现栈空的情况

C. 插入操作更容易实现 D. 删除操作更容易实现

09. 设链表不带头结点且所有操作均在表头进行，则下列最不适合作为链栈的是（ ）。

A. 只有表头结点指针，没有表尾指针的双向循环链表

B. 只有表尾结点指针，没有表头指针的双向循环链表

C. 只有表头结点指针，没有表尾指针的单向循环链表

D. 只有表尾结点指针，没有表头指针的单向循环链表

10. 向一个栈顶指针为 top 的链栈（不带头结点）中插入一个 x 结点，则执行（ ）。

A. top->next=x B. x->next=top->next; top->next=x

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

C. x->next=top; top=x　　D. x->next=top; top=top->next

11. 链栈（不带头结点）执行 pop 操作，并将出栈的元素存在 x 中，应该执行（　）。

A. x=top; top=top->next

B. x=top->data

C. top=top->next; x=top->data

D. x=top->data; top=top->next

12. 经过以下栈的操作后，变量 x 的值为（　）。

```
InitStack(st); Push(st, a); Push(st, b); Pop(st, x); GetTop(st, x);
```

A. a　　B. b　　C. NULL　　D. false

13. 3 个不同元素依次入栈，能得到（　）种不同的出栈序列。

A. 4　　B. 5　　C. 6　　D. 7

14. 设 a, b, c, d, e, f 以所给的次序入栈，若在入栈操作时，允许出栈操作，则下面不会出现的出栈序列为（　）。

A. *fedcba*　　B. *bcafed*　　C. *dcefba*　　D. *cabdef*

15. 4 个元素依次入栈的次序为 a, b, c, d，则以 c, d 开头的出栈序列的个数为（　）。

A. 1　　B. 2　　C. 3　　D. 4

16. 用 S 表示入栈操作，用 X 表示出栈操作，若元素的入栈顺序是 1234，为了得到 1342 的出栈顺序，相应的 S 和 X 的操作序列为（　）。

A. SXSXSSXX　　B. SSSXXSXX　　C. SXSSXXSX　　D. SXSSXSXX

17. 若栈的输入序列是 $1, 2, 3, \cdots, n$，输出序列的第一个元素是 n，则第 i 个输出元素是（　）。

A. 不确定　　B. $n-i$　　C. $n-i-1$　　D. $n-i+1$

18. 若栈的输入序列是 $1, 2, 3, \cdots, n$，输出序列的第一个元素是 i，则第 j 个输出元素是（　）。

A. $i-j-1$　　B. $i-j$　　C. $j-i+1$　　D. 不确定

19. 某栈的输入序列为 a, b, c, d，下面的 4 个序列中，不可能为其输出序列的是（　）。

A. a, b, c, d　　B. c, b, d, a　　C. d, c, a, b　　D. a, c, b, d

20. 若栈的输入序列是 $P_1, P_2, \cdots, P_n$，输出序列是 $1, 2, 3, \cdots, n$，若 $P_3=1$，则 P_1 的值（　）。

A. 可能是 2　　B. 一定是 2　　C. 不可能是 2　　D. 不可能是 3

21. 若栈的输入序列是 $P_1, P_2, \cdots, P_n$，输出序列是 $1, 2, 3, \cdots, n$，若 $P_3=3$，则 P_1 的值（　）。

A. 可能是 2　　B. 不可能是 1　　C. 一定是 1　　D. 一定是 2

22. 已知栈的入栈序列是 1, 2, 3, 4，其出栈序列为 P_1, P_2, P_3, P_4，则 P_2, P_4 不可能是（　）。

A. 2, 4　　B. 2, 1　　C. 4, 3　　D. 3, 4

23. 设栈的初始状态为空，当字符序列“n1_”作为栈的输入时，输出长度为 3，且可用作 C 语言标识符的序列有（　）个。

A. 4　　B. 5　　C. 3　　D. 6

24. 采用共享栈的好处是（　）。

A. 减少存取时间，降低发生上溢的可能

B. 节省存储空间，降低发生上溢的可能

C. 减少存取时间，降低发生下溢的可能

D. 节省存储空间，降低发生下溢的可能

25. 设有一个顺序共享栈 Share[0:n-1]，其中第一个栈顶指针 top1 的初值为-1，第二个栈顶指针 top2 的初值为 n，则判断共享栈满的条件是（　）。

配套课程公众号[考机研]
永久联系微信:4550060

A. top2-top1==1　　B. top1-top2==1

C. top1==top2　　D. 都不对

26. 【2009统考真题】设栈S和队列Q的初始状态均为空，元素*abcdefg*依次进入栈S。若每个元素出栈后立即进入队列Q，且7个元素出队的顺序是*bdcfeag*，则栈S的容量至少是（　）。

A. 1　　B. 2　　C. 3　　D. 4

27. 【2010统考真题】若元素*a*, *b*, *c*, *d*, *e*, *f*依次入栈，允许入栈、出栈操作交替进行，但不允许连续3次进行出栈操作，不可能得到的出栈序列是（　）。

A. *dcebfa*　　B. *cbdaef*　　C. *bcaefd*　　D. *afedcb*

28. 【2011统考真题】元素*a*, *b*, *c*, *d*, *e*依次进入初始为空的栈中，若元素入栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素*d*开头的序列个数是（　）。

A. 3　　B. 4　　C. 5　　D. 6

29. 【2013统考真题】一个栈的入栈序列为1, 2, 3, $\cdots$, n，出栈序列是$P_1, P_2, P_3, \cdots, P_n$。若$P_2=3$，则$P_3$可能取值的个数是（　）。

A. $n-3$　　B. $n-2$　　C. $n-1$　　D. 无法确定

30. 【2020统考真题】对空栈S进行Push和Pop操作，入栈序列为*a*, *b*, *c*, *d*, *e*，经过Push、Push、Pop、Push、Pop、Push、Push、Pop操作后得到的出栈序列是（　）。

A. *b*, *a*, *c*　　B. *b*, *a*, *e*　　C. *b*, *c*, *a*　　D. *b*, *c*, *e*

31. 【2022统考真题】给定有限符号集S，in和out均为S中所有元素的任意排列。对于初始为空的栈ST，下列叙述中，正确的是（　）。

A. 若in是ST的入栈序列，则不能判断out是否为其可能的出栈序列

B. 若out是ST的出栈序列，则不能判断in是否为其可能的入栈序列

C. 若in是ST的入栈序列，out是对应in的出栈序列，则in与out一定不同

D. 若in是ST的入栈序列，out是对应in的出栈序列，则in与out可能互为倒序

二、综合应用题

01. 有5个元素，其入栈次序为*A*, *B*, *C*, *D*, *E*，在各种可能的出栈次序中，第一个出栈元素为*C*且第二个出栈元素为*D*的出栈序列有哪几个？

02. 若元素的入栈序列为*A*, *B*, *C*, *D*, *E*，运用栈操作，能否得到出栈序列*B*, *C*, *A*, *E*, *D*和*D*, *B*, *A*, *C*, *E*？为什么？

03. 栈的初态和终态均为空，以I和O分别表示入栈和出栈，则出入栈的操作序列可表示为由I和O组成的序列，可以操作的序列称为合法序列，否则称为非法序列。

1）下面所示的序列中哪些是合法的？

A. IOIIOIOO　　B. IOOIOIIO　　C. IIIOIOIO　　D. IIIOOIOO

2）通过对1）的分析，写出一个算法，判定所给的操作序列是否合法。若合法，返回true，否则返回false（假定被判定的操作序列已存入一维数组中）。

04. 设单链表的表头指针为*L*，结点结构由data和next两个域构成，其中data域为字符型。试设计算法判断该链表的全部n个字符是否中心对称。例如xyx、xyyx都是中心对称。

05. 设有两个栈S1、S2都采用顺序栈方式，并共享一个存储区[0,…,maxsize-1]，为了尽量利用空间，减少溢出的可能，可采用栈顶相向、迎面增长的存储方式。试设计S1、S2有关入栈和出栈的操作算法。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

3.1.5　答案与解析

一、单项选择题

01．B

栈和队列的逻辑结构都是相同的，都属于线性结构，只是它们对数据的运算不同。

02．C

首先栈是一种线性表，所以选项 B、D 错。按存储结构的不同可分为顺序栈和链栈，但不可以把栈局限在某种存储结构上，所以选项 A 错。栈和队列都是限制存取点的线性结构。

03．B

基本操作是指该结构最核心、最基本的运算，其他较复杂的操作可通过基本操作实现。删除栈底元素不属于栈的基本运算，但它可以通过调用栈的基本运算求得。

04．C

数组下标范围为 0～n-1，初始时 top 为-1，第一个元素入栈后，top 为 0，即 top 指向栈顶元素。栈向高地址方向增长，所以入栈时应先将指针 top 加 1，然后存入元素 x，选项 C 正确。

05．B

数组下标范围为 1～n，初始时 top 为 1，表示 top 指向栈顶元素的下一个元素。栈向高地址方向增长，所以入栈时应先存入元素 x，然后将指针 top 加 1，选项 B 正确。

06．A

数组下标范围为 1～n，初始时 top 为 n+1，表示 top 指向栈顶元素。栈向低地址方向增长，所以入栈时应先将指针 top 减 1，然后存入元素 x，A 正确。

07．A

每个元素需要 1 个存储单元，所以每入栈一次 top 加 1，出栈一次 top 减 1。指针 top 的值依次为 1001H, 1002H, 1001H, 1002H, 1001H, 1002H, 1001H, 1002H。

08．A

顺序栈采用数组存储，数组的大小是固定的，不能动态地分配大小。和顺序栈相比，链栈的最大优势在于它可以动态地分配存储空间。

09．C

对于双向循环链表，不管是表头指针还是表尾指针，都可以很方便地找到表头结点，方便在表头做插入或删除操作。而循环单链表通过尾指针可以很方便地找到表头结点，但通过头指针找尾结点需要遍历一次链表。对于选项 C，插入和删除结点后，找尾结点所需的时间为 $O(n)$。

10．C

链栈采用不带头结点的单链表表示时，入栈操作在首部插入一个结点 x（x->next=top），插入完后需将 top 指向该插入的结点 x。请思考当链栈存在头结点时的情况。

11．D

这里假设栈顶指针指向的是栈顶元素，所以选择选项 D；而选项 A 中首先将 top 指针赋给了 x，错误；选项 B 中没有修改 top 指针的值；选项 C 为 top 指针指向栈顶元素的上一个元素时的答案。

12．A

执行前 3 句后，栈 st 内的值为 a, b，其中 b 为栈顶元素；执行第 4 句后，栈顶元素 b 出栈，x 的值为 b；执行最后一句，读取栈顶元素的值，x 的值为 a。

13．B

配套课程公众号[考机研]
永久联系微信:4550060

当 n 个不同元素入栈时，出栈序列的个数为

$$\frac{1}{n+1}C_{2n}^{n}=\frac{1}{n+1}\frac{(2n)!}{n!\times n!}=\frac{6\times5\times4}{4\times3\times2\times1}=5$$

考题中给出的 n 值不会很大，可以根据栈的特点，若 X_i 已经出栈，则 X_i 前面的尚未出栈的元素一定逆置有序地出栈，因此可采用例举方法。如 a, b, c 依次入栈的出栈序列有 abc, acb, bac, bca, cba。另外，在一些考题中可能会问符合某个特定条件的出栈序列有多少种，比如此题中的问以 b 开头的出栈序列有几种，这种类型的题目一般都使用穷举法。

14. D

根据栈“先进后出”的特点，且在入栈操作的同时允许出栈操作，显然选项 D 中 c 最先出栈，则此时栈内必定为 a 和 b，但因为 a 先于 b 入栈，所以要晚出栈。对于某个出栈的元素，在它之前入栈却晚出栈的元素必定是按逆序出栈的，其余答案均是可能出现的情况。

此题也可采用将各序列逐个代入的方法来确定是否有对应的进出栈序列（类似下题）。

15. A

假设出栈序列为 $cd\ldots$，分析栈的操作序列：a 入栈，b 入栈，c 入栈，c 出栈，d 入栈，d 出栈，此后只能是 b 出栈和 a 出栈一种情况，因此出栈序列只有 $cdba$。

16. D

采用排除法，选项 A, B, C 得到的出栈序列分别为 1243, 3241, 1324。由 1234 得到 1342 的进出栈序列为：1 进，1 出，2 进，3 进，3 出，4 进，4 出，2 出，所以选择选项 D。

17. D

第 n 个元素第一个出栈，说明前 $n-1$ 个元素都已经按顺序入栈，由“先进后出”的特点可知，此时的输出序列一定是输入序列的逆序，所以答案选择选项 D。

18. D

当第 i 个元素第一个出栈时，则 i 之前的元素可以依次排在 i 之后出栈，但剩余的元素可以在此时入栈，并且排在 i 之前的元素出栈，所以第 j 个出栈的元素是不确定的。

19. C

对于 A，可能的顺序是 a 入，a 出，b 入，b 出，c 入，c 出，d 入，d 出。对于选项 B，可能的顺序是 a 入，b 入，c 入，c 出，b 出，d 入，d 出，a 出。对于选项 D，可能的顺序是 a 入，a 出，b 入，c 入，c 出，b 出，d 入，d 出。选项 C 没有对应的序列。

【另解】若出栈序列的第一个元素为 d，则出栈序列只能是 $dcba$。该思想通常也适用于出栈序列的局部分析：如 12345 入栈，问出栈序列 34152 是否正确？如何分析？若第一个出栈元素是 3，则此时 12 必停留在栈中，它们出栈的相对顺序只能是 21，所以 34152 错误。

20. C

入栈序列是 $P_1, P_2, \cdots, P_n$。因为 $P_3=1$，即 P_1, P_2, P_3 连续入栈后，第一个出栈元素是 P_3，说明 P_1, P_2 已经按序入栈，根据先进后出的特点可知，P_2 必定在 P_1 之前出栈，而第二个出栈元素是 2，而此时 P_1 不是栈顶元素，所以 P_1 的值不可能是 2。思考：哪些 P_i 可能是 2？

21. A

假设 P_1 是 1，入栈后立即出栈，P_2 是 2，入栈后立即出栈，P_3 是 3，入栈后立即出栈，得到的序列符合题意。假设 P_1 是 2，P_2 是 1，P_1、P_2 依次入栈后全部出栈，P_3 是 3，入栈后立即出栈，得到的序列符合题意。因此，P_1 既可能是 1，又可能是 2。

22. C

逐个判断每个选项可能的入栈出栈顺序。对于选项 A，可能的顺序是 1 入，1 出，2 入，2 出，

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

3 入，3 出，4 入，4 出。对于选项 B，可能的顺序是 1 入，2 入，3 入，3 出，2 出，4 入，4 出，1 出。对于选项 D，可能的顺序是 1 入，1 出，2 入，3 入，3 出，2 出，4 入，4 出。选项 C 没有对应的序列，因为当 4 在栈中时，意味着前面的所有元素（1, 2, 3）都已在栈中或曾经入过栈，此时若 4 第二个出栈，即栈中还有两个元素，且这两个元素是有序的（对应入栈顺序），只能为(1, 2), (1, 3), (2, 3)，若是序列(1, 2)，则 3 已在 p_1 位置出栈，不可能再在 p_4 位置出栈，若是(1, 3)和(2, 3)这种情况中的任意一种，则 3 一定是下一个出栈元素，即 p_3 一定是 3，所以 p_4 不可能是 3。

【另解】对于 C，p_2 为最后一个入栈元素 4，则只有 p_1 或 p_3 出栈的元素有可能为 3（请读者分两种情况自行思考），而 p_4 绝不可能为 3。读者在解答此类题时，一定要注意出栈序列中的“最后一个入栈元素”，这样可以节省答题的时间。

23．C

标识符只能以英文字母或下画线开头，而不能以数字开头。于上，由 n、1、_三个字符组合成的标识符有 n1_、n_1、_1n 和_n1 四种。第一种：n 入栈再出栈，1 入栈再出栈，_入栈再出栈。第二种：n 入栈再出栈，1 入栈，_入栈，_出栈，1 出栈。第三种：n 入栈，1 入栈，_入栈，_出栈，1 出栈，n 出栈。而根据栈的操作特性，_n1 这种情况不可能出现。

24．B

上溢是指存储器满，还往里写；下溢是指存储器空，还往外读。为了解决上溢，可给栈分配很大的存储空间，但这样又会造成存储空间的浪费。共享栈的提出就是为了在解决上溢的基础上节省存储空间，将两个栈放在同一段更大的存储空间内，这样，当一个栈的元素增加时，可使用另一个栈的空闲空间，从而降低发生上溢的可能性。

25．A

这种情况就是前面我们所描述的，详细内容请参见本节考点精析部分对共享栈的讲解。另外，读者可以思考当 top1 的初值为 0，top2 的初值为 n-1 时栈满的条件。

注　意

栈顶、队头与队尾的指针的定义是不唯一的，做题时务必仔细审题和思考。

26．C

时刻注意栈的特点是先进后出，下表是出入栈的详细过程。

序号	说明	栈内	栈外	序号	说明	栈内	栈外
1	*a* 入栈	*a*		8	*e* 入栈	*ae*	*bdc*
2	*b* 入栈	*ab*		9	*f* 入栈	*aef*	*bdc*
3	*b* 出栈	*a*	*b*	10	*f* 出栈	*ae*	*bdcf*
4	*c* 入栈	*ac*	*b*	11	*e* 出栈	*a*	*bdcfe*
5	*d* 入栈	*acd*	*b*	12	*a* 出栈		*bdcfea*
6	*d* 出栈	*ac*	*bd*	13	*g* 入栈	*g*	*bdcfea*
7	*c* 出栈	*a*	*bdc*	14	*g* 出栈		*bdcfeag*

栈内的最大深度为 3，所以栈 S 的容量至少是 3。

【另解】因为元素的出队顺序和入队顺序相同，所以元素的出栈顺序就是 *b*, *d*, *c*, *f*, *e*, *a*, *g*，因此元素的出入栈次序为 Push(S, *a*), Push(S, *b*), Pop(S, *b*), Push(S, *c*), Push(S, *d*), Pop(S, *d*), Pop(S, *c*), Push(S, *e*), Push(S, *f*), Pop(S, *f*), Pop(S, *e*), Pop(S, *a*), Push(S, *g*), Pop(S, *g*)。初始所需容量为 0，每做一次 Push 操作，容量加 1；每做一次 Pop 操作，容量减 1，记录的容量最大值为 3。

配套课程公众号[考机研]
永久联系微信:4550060

27. D

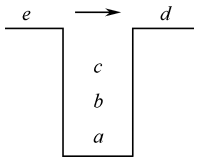
选项A由a入，b入，c入，d入，d出，c出，e入，e出，b出，f入，f出，a出得到；选项B由a入，b入，c入，c出，b出，d入，d出，a出，e入，e出，f入，f出得到；选项C由a入，b入，b出，c入，c出，a出，d入，e入，e出，f入，f出，d出得到；选项D由a入，a出，b入，c入，d入，e入，f入，f出，e出，d出，c出，b出得到，但题意要求不允许连续3次出栈操作，选项D不符。

【另解】先入栈的元素后出栈，入栈顺序为a, b, c, d, e, f，所以连续出栈时的子序列必然是按字母表逆序的，若出栈序列中出现了长度大于或等于3的连续逆序子序列，则为所选序列。

28. B

d第一个出栈，则c, b, a出栈的相对顺序是确定的，出栈顺序必为$d_c_b_a_$，e的顺序不定，在任意一个“_”上都有可能。

【另解】d首先出栈，则abc停留在栈中，此时栈的状态如下图所示。



此时可以有如下4种操作：①e入栈后出栈，则出栈序列为$decba$；②c出栈，e入栈后出栈，出栈序列为$dceba$；③cb出栈，e入栈后出栈，出栈序列为$dcbea$；④cba出栈，e入栈后出栈，出栈序列为$dcbae$。思路和上面其实一样。

29. C

显然，3之后的$4, 5, \cdots, n$都是P_3可取的数（一直入栈直到该数入栈后马上出栈）。接下来分析1和2是否可取：P_1可以是3之前入栈的数（可能是1或2），也可以是4，当$P_1=1$时，P_3可取2；当$P_1=2$时，P_3可取1。因此，p_3可能取除3外的所有数，个数为$n-1$。

30. D

按题意，出入栈操作的过程如下：

操作	栈内元素	出栈元素
Push	a	
Push	$a\ b$	
Pop	a	b
Push	$a\ c$	
Pop	a	c
Push	$a\ d$	
Push	$a\ d\ e$	
Pop	$a\ d$	e

因此，出栈序列为b, c, e。

31. D

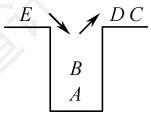
通过模拟出入栈操作，可以判断入栈序列in和出栈序列out是否合法。因此，已知in序列可以判断out序列是否为可能的出栈序列；已知out序列也可以判断in序列是否为可能的入栈序列，选项A和B错误。若每个元素入栈后立即出栈，则in序列和out序列相同，选项C错误。若所有元素都入栈后才依次出栈，则in序列和out序列互为倒序，选项D正确。

配套课程公众号[考机研]
永久联系微信:4550060

二、综合应用题

01.【解答】

CD 出栈后的状态如下图所示。



此时有如下 3 种操作：①*E* 入栈后出栈，出栈序列为 *CDEBA*；②*B* 出栈，*E* 入栈后出栈，出栈序列为 *CDBEA*；③*B* 出栈，*A* 出栈，*E* 入栈后出栈，出栈序列为 *CDBAE*。

所以，以 *CD* 开头的出栈序列有 *CDEBA*、*CDBEA*、*CDBAE* 三种。

02.【解答】

能得到出栈序列 *BCAED*。可由 *A* 入，*B* 入，*B* 出，*C* 入，*C* 出，*A* 出，*D* 入，*E* 入，*E* 出，*D* 出得到。不能得到出栈序列 *DBACE*。若出栈序列以 *D* 开头，说明在 *D* 之前的入栈元素是 *A*、*B* 和 *C*，三个元素中 *C* 是栈顶元素，*B* 和 *A* 不可能早于 *C* 出栈，所以不可能得到出栈序列 *DBACE*。

03.【解答】

1）选项 A、D 合法，而选项 B、C 不合法。在 B 中，先入栈 1 次，再连续出栈 2 次，错误。在选项 C 中，入栈和出栈次数不一致，会导致最终的栈不空。选项 A、D 均为合法序列，请自行模拟。注意，在操作过程中，入栈次数一定大于或等于出栈次数；结束时，栈一定为空。

2）设被判定的操作序列已存入一维数组 *A* 中。算法的基本设计思想：依次逐一扫描入栈出栈序列（由“I”和“O”组成的字符串），每扫描至任意一个位置均需检查出栈次数（“O”的个数）是否小于入栈次数（“I”的个数），若大于则为非法序列。扫描结束后，再判断入栈和出栈次数是否相等，若不相等则不合题意，为非法序列。

本题代码如下：

```
bool Judge(char A[ ]){
    int i=0;
    int j=k=0;                          //i 为下标，j 和 k 分别为字母 I 和 O 的个数
    while(A[i]!='\0'){                  //未到字符数组尾
        switch(A[i]){
            case 'I': j++; break;       //入栈次数增 1
            case 'O': k++;
                if(k>j){printf("序列非法\n"); exit(0); }
        }
        i++;                            //不论 A[i]是 I 还是 O，指针 i 均后移
    }
    if(j!=k){
        printf("序列非法\n");
        return  false;
    }
    else{
        printf("序列合法\n");
        return  true;
    }
}
```

【另解】入栈后，栈内元素个数加 1；出栈后，栈内元素个数减 1，因此可将判定一组出入栈序列是否合法转化为一组由+1、−1 组成的序列，它的任意前缀子序列的累加和不小于 0（每次出栈或入栈操作后判断）则合法；否则非法。

04.【解答】

算法思想：使用栈来判断链表中的数据是否中心对称。让链表的前一半元素依次入栈。在处

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

理链表的后一半元素时，当访问到链表的一个元素后，就从栈中弹出一个元素，对两个元素进行比较，若相等，则将链表中的下一个元素与栈中再弹出的元素进行比较，直至链表到尾。这时若栈是空栈，则得出链表中心对称的结论；否则，当链表中的一个元素与栈中弹出元素不等时，结论为链表非中心对称，结束算法的执行。

本题代码如下：

```
int dc(LinkList L,int n){
    int i;
    char s[n/2];                          //s 字符栈
    LNode *p=L->next;                     //工作指针 p，指向待处理的当前元素
    for(i=0;i<n/2;i++){                   //链表前一半元素入栈
        s[i]=p->data;
        p=p->next;
    }
    i--;                                  //恢复最后的 i 值
    if(n%2==1)                            //若 n 是奇数，后移过中心结点
        p=p->next;
    while(p!=NULL&&s[i]==p->data){        //检测是否中心对称
        i--;                              //i 充当栈顶指针
        p=p->next;
    }
    if(i==-1)                             //栈为空栈
        return  1;                        //链表中心对称
    else
        return  0;                        //链表中心不对称
}
```

算法先将“链表的前一半”元素（字符）入栈。当 n 为偶数时，前一半和后一半的个数相同；当 n 为奇数时，链表中心结点字符不必比较，移动链表指针到下一字符开始比较。比较过程中遇到不相等时，立即退出 while 循环，不再进行比较。

本题也可以先将单链表中的元素全部入栈，然后扫描单链表 L 并比较，直到比较到单链表 L 尾为止，但算法需要两次扫描单链表 L，效率不及上述算法高。

05.【解答】

两个栈共享向量空间，将两个栈的栈底设在向量两端，初始时，S1 栈顶指针为-1，S2 栈顶指针为 maxsize。两个栈顶指针相邻时为栈满。两个栈顶相向、迎面增长，栈顶指针指向栈顶元素。

本题代码如下：

```
#define  maxsize   100              //两个栈共享顺序存储空间所能达到的最多元素数，
                                    //初始化为 100
#define  elemtp int                 //假设元素类型为整型
typedef  struct{
    elemtp stack[maxsize];          //栈空间
    int   top[2];                   //top 为两个栈顶指针
}stk;
stk   s;                            //s 是如上定义的结构类型变量，为全局变量
```

本题的关键在于，两个栈入栈和出栈时的栈顶指针的计算。S1 栈是通常意义下的栈；而 S2 栈入栈操作时，其栈顶指针左移（减 1），出栈时，栈顶指针右移（加 1）。

此外，对于所有栈的操作，都要注意“入栈判满、出栈判空”的检查。

（1）入栈操作

代码如下：

```
int push(int i, elemtp x){
//入栈操作。i 为栈号，i=0 表示左边的 S1 栈，i=1 表示右边的 S2 栈，x 是入栈元素
```

配套课程公众号[考机研]
永久联系微信:4550060

```
    //入栈成功返回 1，否则返回 0
        if(i<0||i>1){
            printf("栈号输入不对");
            exit(0);
        }
        if(s.top[1]-s.top[0]==1){
            printf("栈已满\n");
            return  0;
        }
        switch(i){
            case 0: s.stack[++s.top[0]]=x; return  1; break;
            case 1: s.stack[--s.top[1]]=x; return  1;
        }
    }
```

（2）出栈操作

代码如下：

```
    elemtp pop(int i){
    //出栈算法。i 代表栈号，i=0 时为 S1 栈，i=1 时为 S2 栈
    //出栈成功返回出栈元素，否则返回-1
        if(i<0||i>1){
            printf("栈号输入错误\n");
            exit(0);
        }
        switch(i){
            case 0:
                if(s.top[0]==-1){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[0]--];
                break;
            case 1:
                if(s.top[1]==maxsize){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[1]++];
                break;
        }//switch
    }
```

3.2 队列

3.2.1 队列的基本概念

1．队列的定义

队列（Queue）简称队，也是一种操作受限的线性表，只允许在表的一端进行插入，而在表的另一端进行删除。向队列中插入元素称为入队或进队；删除元素称为出队或离队。这和我们日常生活中的排队是一致的，最早排队的也是最早离队的，其操作的特性是先进先出（First In First Out，FIFO），如图 3.5 所示。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

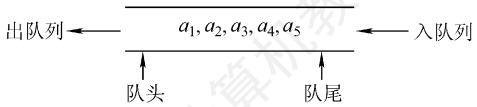


图 3.5 队列示意图

队头（Front）：允许删除的一端，也称队首。

队尾（Rear）：允许插入的一端。

空队列：不含任何元素的空表。

2. 队列常见的基本操作

`InitQueue(&Q)`：初始化队列，构造一个空队列 `Q`。

`QueueEmpty(Q)`：判队列空，若队列 `Q` 为空返回 `true`，否则返回 `false`。

`EnQueue(&Q,x)`：入队，若队列 `Q` 未满，将 `x` 加入，使之成为新的队尾。

`DeQueue(&Q,&x)`：出队，若队列 `Q` 非空，删除队首元素，并用 `x` 返回。

`GetHead(Q,&x)`：读队首元素，若队列 `Q` 非空，则将队首元素赋值给 `x`。

需要注意的是，栈和队列是操作受限的线性表，因此不是任何对线性表的操作都可以作为栈和队列的操作。比如，不可以随便读取栈或队列中间的某个数据。

3.2.2 队列的顺序存储结构

1. 队列的顺序存储

队列的顺序实现是指分配一块连续的存储单元存放队列中的元素，并附设两个指针：队首指针 `front` 指向队首元素，队尾指针 `rear` 指向队尾元素的下一个位置（不同教材对 `front` 和 `rear` 的定义可能不同，例如，可以让 `rear` 指向队尾元素、`front` 指向队首元素。对于不同的定义，出入队的操作是不同的，本节后面有一些相关的习题，读者可以结合习题思考）。

队列的顺序存储类型可描述为

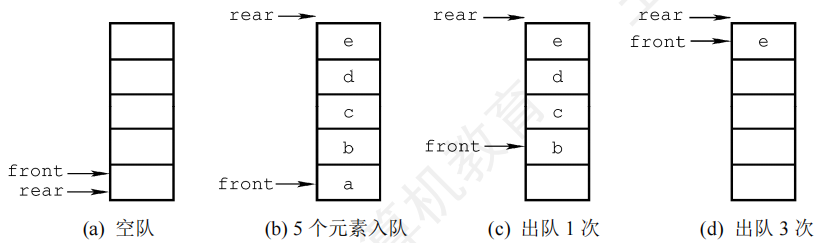
```
#define MaxSize 50                  //定义队列中元素的最大个数
   typedef struct{
   ElemType data[MaxSize];          //用数组存放队列元素
   int front,rear;                  //队首指针和队尾指针
}SqQueue;
```

初始时：`Q.front=Q.rear=0`。

入队操作：队不满时，先送值到队尾元素，再将队尾指针加 1。

出队操作：队不空时，先取队首元素值，再将队首指针加 1。

图 3.6(a)所示为队列的初始状态，有 `Q.front==Q.rear==0` 成立，该条件可以作为队列判空的条件。



(a) 空队 (b) 5 个元素入队 (c) 出队 1 次 (d) 出队 3 次

图 3.6 队列的操作

但能否用 `Q.rear==MaxSize` 作为队列满的条件呢？显然不能，图 3.6(d)中，队列中仅有一

配套课程公众号[考机研]
永久联系微信:4550060

个元素，但仍满足该条件。这时入队出现“上溢出”，但这种溢出并不是真正的溢出，在 data 数组中依然存在可以存放元素的空位置，所以是一种“假溢出”。

2．循环队列

上面指出了顺序队列“假溢出”的问题，这里引出循环队列的概念。将顺序队列臆造为一个环状的空间，即把存储队列元素的表从逻辑上视为一个环，称为循环队列。当队首指针 Q.front=MaxSize-1 后，再前进一个位置就自动到 0，这可以利用除法取模运算（%）来实现。

命题追踪 ▶▶ 特定条件下循环队列队头/队尾指针的初值（2011）

初始时：Q.front=Q.rear=0。

队首指针进 1：Q.front=(Q.front+1)%MaxSize。

队尾指针进 1：Q.rear=(Q.rear+1)%MaxSize。

队列长度：(Q.rear+MaxSize-Q.front)%MaxSize。

出入队时：指针都按顺时针方向进 1（如图 3.7 所示）。

命题追踪 ▶▶ 特定条件下循环队列队空/队满的判断条件（2014）

那么，循环队列队空和队满的判断条件是什么呢？显然，队空的条件是 Q.front==Q.rear。若入队元素的速度快于出队元素的速度，则队尾指针很快就会赶上队首指针，如图 3.7(d1)所示，此时可以看出队满时也有 Q.front==Q.rear。循环队列出入队示意图如图 3.7 所示。

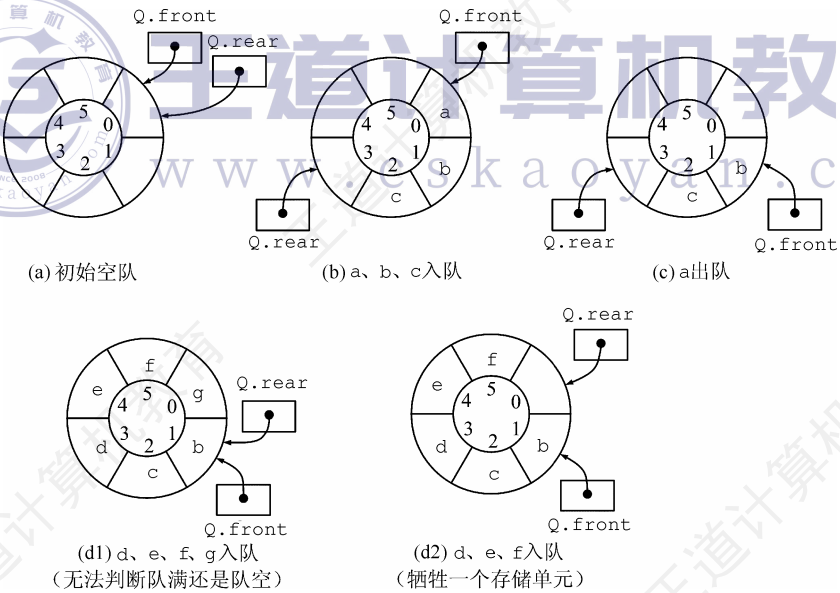


图 3.7　循环队列出入队示意图

为了区分是队空还是队满的情况，有三种处理方式：

1）牺牲一个单元来区分队空和队满，入队时少用一个队列单元，这是一种较为普遍的做法，约定以“队首指针在队尾指针的下一位置作为队满的标志”，如图 3.7(d2)所示。

队满条件：(Q.rear+1)%MaxSize==Q.front。

队空条件：Q.front==Q.rear。

队列中元素的个数：(Q.rear-Q.front+MaxSize)%MaxSize。

2）类型中增设 size 数据成员，表示元素个数。若删除成功，则 size 减 1，若插入成功，

配套课程公众号[考机研]
永久联系微信:4550060

则 size 加 1，队空时 Q.size==0；队满时 Q.size==MaxSize，两种情况都有 Q.front==Q.rear。

3）类型中增设 tag 数据成员，以区分是队满还是队空。删除成功置 tag=0，若导致 Q.front==Q.rear，则为队空；插入成功置 tag=1，若导致 Q.front==Q.rear，则为队满。

3．循环队列的操作

（1）初始化

```
void InitQueue(SqQueue &Q){
    Q.rear=Q.front=0;                    //初始化队首、队尾指针
}
```

（2）判队空

```
bool isEmpty(SqQueue Q){
    if(Q.rear==Q.front)                  //队空条件
        return true;
    else
        return  false;
}
```

（3）入队

```
bool EnQueue(SqQueue &Q,ElemType x){
    if((Q.rear+1)%MaxSize==Q.front)      //队满则报错
        return false;
    Q.data[Q.rear]=x;
    Q.rear=(Q.rear+1)%MaxSize;           //队尾指针加 1 取模
    return  true;
}
```

（4）出队

```
bool DeQueue(SqQueue &Q,ElemType &x){
    if(Q.rear==Q.front)                  //队空则报错
        return false;
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;         //队首指针加 1 取模
    return  true;
}
```

3.2.3 队列的链式存储结构

1．队列的链式存储

命题追踪 ▶▶ 根据需求分析队列适合的存储结构（2019）

队列的链式表示称为链式队列，它实际上是一个同时有队首指针和队尾指针的单链表，如图 3.8 所示。队首指针指向队头结点，队尾指针指向队尾结点，即单链表的最后一个结点。

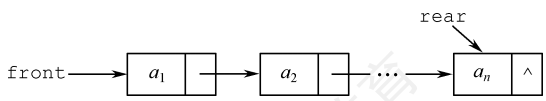


图 3.8 不带队头结点的链式队列

队列的链式存储类型可描述为

```
typedef struct LinkNode{                 //链式队列结点
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
typedef struct{                          //链式队列
    LinkNode *front,*rear;               //队列的队头和队尾指针
}LinkQueue;
```

不带头结点时，当 Q.front==NULL 且 Q.rear==NULL 时，链式队列为空。

命题追踪 ▶▶ 链式队列队空的判断（2019）

入队时，建立一个新结点，将新结点插入到链表的尾部，并让 Q.rear 指向这个新插入的结点（若原队列为空队，则令 Q.front 也指向该结点）。出队时，首先判断队是否为空，若不空，则取出队首元素，将其从链表中删除，并让 Q.front 指向下一个结点（若该结点为最后一个结点，则置 Q.front 和 Q.rear 都为 NULL）。

不难看出，不带头结点的链式队列在操作上往往比较麻烦，因此通常将链式队列设计成一个带头结点的单链表，这样插入和删除操作就统一了，如图 3.9 所示。

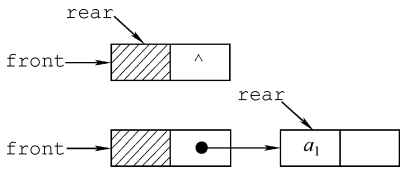


图 3.9　带队头结点的链式队列

用单链表表示的链式队列特别适合于数据元素变动比较大的情形，而且不存在队列满且产生溢出的问题。另外，假如程序中要使用多个队列，与多个栈的情形一样，最好使用链式队列，这样就不会出现存储分配不合理和“溢出”的问题。

2．链式队列的基本操作

命题追踪 ▶▶ 链式队列出队/入队操作的基本过程（2019）

（1）初始化

```
void InitQueue(LinkQueue &Q){   //初始化带头结点的链式队列
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));//建立头结点
    Q.front->next=NULL;         //初始为空
}
```

（2）判队空

```
bool IsEmpty(LinkQueue Q){
    if(Q.front==Q.rear)         //判空条件
        return true;
    else
        return false;
}
```

（3）入队

```
void EnQueue(LinkQueue &Q,ElemType x){
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode)); //创建新结点
    s->data=x;
    s->next=NULL;
    Q.rear->next=s;             //插入链尾
    Q.rear=s;                   //修改尾指针
}
```

（4）出队

```
bool DeQueue(LinkQueue &Q,ElemType &x){
    if(Q.front==Q.rear)
        return  false;          //空队
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        LinkNode *p=Q.front->next;
        x=p->data;
        Q.front->next=p->next;
        if(Q.rear==p)
            Q.rear=Q.front;              //若原队列中只有一个结点，删除后变空
        free(p);
        return  true;
    }
```

3.2.4 双端队列

双端队列是指允许两端都可以进行插入和删除操作的线性表，如图 3.10 所示。双端队列两端的地位是平等的，为了方便理解，将左端也视为前端，右端也视为后端。

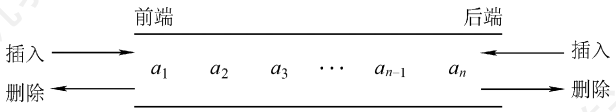


图 3.10 双端队列

在双端队列入队时，前端进的元素排列在队列中后端进的元素的前面，后端进的元素排列在队列中前端进的元素的后面。在双端队列出队时，无论是前端还是后端出队，先出的元素排列在后出的元素的前面。思考：如何由入队序列 *a*, *b*, *c*, *d* 得到出队序列 *d*, *c*, *a*, *b*?

命题追踪 ▶ 双端队列出队/入队操作模拟（2010、2021）

输出受限的双端队列：允许在一端进行插入和删除，但在另一端只允许插入的双端队列称为输出受限的双端队列，如图 3.11 所示。

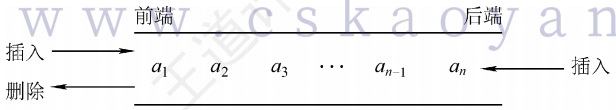


图 3.11 输出受限的双端队列

输入受限的双端队列：允许在一端进行插入和删除，但在另一端只允许删除的双端队列称为输入受限的双端队列，如图 3.12 所示。若限定双端队列从某个端点插入的元素只能从该端点删除，则该双端队列就蜕变为两个栈底相邻接的栈。

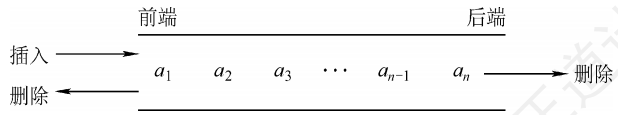


图 3.12 输入受限的双端队列

例 设有一个双端队列，输入序列为 1, 2, 3, 4，试分别求出以下条件的输出序列。

（1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列。

（2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列。

（3）既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的输出序列。

解：先看输入受限的双端队列，如图 3.13 所示。假设 end1 端输入 1, 2, 3, 4，则 end2 端的输出相当于队列的输出，即 1, 2, 3, 4；而 end1 端的输出相当于栈的输出，$n=4$ 时仅通过 end1 端有 14 种输出序列（由 Catalan 公式得出），仅通过 end1 端不能得到的输出序列有 4! – 14 = 10 种：

配套课程公众号[考机研]
永久联系微信:4550060

1, 4, 2, 3　　2, 4, 1, 3　　3, 4, 1, 2　　3, 1, 4, 2　　3, 1, 2, 4
4, 3, 1, 2　　4, 1, 3, 2　　4, 2, 3, 1　　4, 2, 1, 3　　4, 1, 2, 3

通过 end1 和 end2 端混合输出，可以输出这 10 种中的 8 种，参看下表。其中，S_L, X_L 分别代表 end1 端的入队和出队，X_R 代表 end2 端的出队。

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_LX_RS_LS_LX_LX_RX_R$	3, 1, 2, 4	$S_LS_LS_LX_LS_LX_RX_RX_R$
2, 4, 1, 3	$S_LS_LX_LS_LS_LX_LX_RX_R$	4, 1, 2, 3	$S_LS_LS_LS_LX_LX_RX_RX_R$
3, 4, 1, 2	$S_LS_LS_LX_LS_LX_LX_RX_R$	4, 1, 3, 2	$S_LS_LS_LS_LX_LX_RX_LX_R$
3, 1, 4, 2	$S_LS_LS_LX_LX_RS_LX_LX_R$	4, 3, 1, 2	$S_LS_LS_LS_LX_LX_LX_RX_R$

剩下两种是不能通过输入受限的双端队列输出的，即 4, 2, 3, 1 和 4, 2, 1, 3。

再看输出受限的双端队列，如图 3.14 所示。假设 end1 端和 end2 端都能输入，仅 end2 端可以输出。若都从 end2 端输入，就是一个栈了。当输入序列为 1, 2, 3, 4 时，输出序列有 14 种。对于其他 10 种不能得到的输出序列，交替从 end1 和 end2 端输入，还可以输出其中 8 种。设 S_L 代表 end1 端的输入，S_R、X_R 分别代表 end2 端的输入和输出，则可能的输出序列见下表。

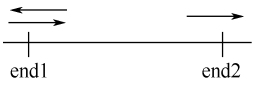


图 3.13　例题中输入受限的双端队列

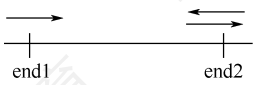


图 3.14　例题中输出受限的双端队列

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_LX_RS_LS_LS_RX_RX_R$	3, 1, 2, 4	$S_LS_LS_RX_RX_RS_LX_RX_R$
2, 4, 1, 3	$S_LS_RX_RS_LS_RX_RX_R$	4, 1, 2, 3	$S_LS_LS_LS_RX_RX_RX_R$
3, 4, 1, 2	$S_LS_LS_RX_RS_RX_RX_R$	4, 2, 1, 3	$S_LS_RS_LS_RX_RX_RX_R$
3, 1, 4, 2	$S_LS_LS_RX_RX_RS_RX_R$	4, 3, 1, 2	$S_LS_LS_RS_RX_RX_RX_R$

通过输出受限的双端队列不能得到的两种输出序列是 4, 1, 3, 2 和 4, 2, 3, 1。

综上所述：

1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的是 4, 1, 3, 2。

2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的是 4, 2, 1, 3。

3）既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的是 4, 2, 3, 1。

提　示

实际双端队列的考题不会这么复杂，通常仅判断序列是否满足题设条件，代入验证即可。

3.2.5　本节试题精选

一、单项选择题

01. 栈和队列的主要区别在于（　）。

A. 它们的逻辑结构不一样　　B. 它们的存储结构不一样

C. 所包含的元素不一样　　D. 插入、删除操作的限定不一样

02. 队列的“先进先出”特性是指（　）。

I. 最后插入队列中的元素总是最后被删除

II. 当同时进行插入、删除操作时，总是插入操作优先

配套课程公众号[考机研]
永久联系微信:4550060

III. 每当有删除操作时，总要先做一次插入操作

IV. 每次从队列中删除的总是最早插入的元素

A. I　　B. I和IV　　C. II和III　　D. IV

03. 允许对队列进行的操作有（　）。

A. 对队列中的元素排序　　B. 取出最近入队的元素

C. 在队列元素之间插入元素　　D. 删除队首元素

04. 一个队列的入队顺序是1, 2, 3, 4，则出队的输出顺序是（　）。

A. 4, 3, 2, 1　　B. 1, 2, 3, 4　　C. 1, 4, 3, 2　　D. 3, 2, 4, 1

05. 循环队列存储在数组A[0…n]中，入队时的操作为（　）。

A. rear=rear+1　　B. rear=(rear+1) mod (n-1)

C. rear=(rear+1) mod n　　D. rear=(rear+1) mod (n+1)

06. 已知循环队列的存储空间为数组A[21]，front指向队首元素的前一个位置，rear指向队尾元素，假设当前front和rear的值分别为8和3，则该队列的长度为（　）。

A. 5　　B. 6　　C. 16　　D. 17

07. 若用数组A[0…5]实现循环队列，且当前rear和front的值分别为1和5，当从队列中删除一个元素，再加入两个元素后，rear和front的值分别为（　）。

A. 3和4　　B. 3和0　　C. 5和0　　D. 5和1

08. 假设用数组Q[MaxSize]实现循环队列，队首指针front指向队首元素的前一位置，队尾指针rear指向队尾元素，则判断该队列为空的条件是（　）。

A. Q.rear==(Q.front+1)%MaxSize

B. (Q.rear+1)%MaxSize==Q.front+1

C. (Q.rear+1)%MaxSize==Q.front

D. Q.rear==Q.front

09. 假设循环队列Q[MaxSize]的队首指针为front，队尾指针为rear，队列的最大容量为MaxSize，此外，该队列再没有其他数据成员，则判断该队列已满足条件是（　）。

A. Q.front==Q.rear　　B. Q.front+Q.rear>=MaxSize

C. Q.front==(Q.rear+1)%MaxSize　　D. Q.rear==(Q.front+1)%MaxSize

10. 假设用A[0…n]实现循环队列，front、rear分别指向队首元素的前一个位置和队尾元素。若用(rear+1)%(n+1)==front作为队满标志，则（　）。

A. 可用front==rear作为队空标志　　B. 队列中最多可有n+1个元素

C. 可用front>rear作为队空标志

D. 可用(front+1)%(n+1)==rear作为队空标志

11. 与顺序队列相比，链式队列的（　）。

A. 优点是队列的长度不受限制

B. 优点是入队和出队时间效率更高

C. 缺点是不能进行顺序访问

D. 缺点是不能根据队首指针和队尾指针计算队列的长度

12. 下列描述的几种链表中，最适合用作队列的是（　）。

A. 带队首指针和队尾指针的循环单链表

B. 带队首指针和队尾指针的非循环单链表

C. 只带队首指针的非循环单链表

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

D. 只带队首指针的循环单链表

13. 下列描述的几种链表中，最不适合用作链式队列的是（　）。

A. 只带队首指针的非循环双链表　　B. 只带队首指针的循环双链表

C. 只带队尾指针的循环双链表　　D. 只带队尾指针的循环单链表

14. 在用单链表实现队列时，队头设在链表的（　）位置。

A. 链头　　B. 链尾　　C. 链中　　D. 以上都可以

15. 用链式存储方式的队列进行删除操作时需要（　）。

A. 仅修改头指针　　B. 仅修改尾指针

C. 头尾指针都要修改　　D. 头尾指针可能都要修改

16. 在一个链式队列中，假设队首指针为 front，队尾指针为 rear，x 所指向的元素需要入队，则需要执行的操作为（　）。

A. front=x, front=front->next

B. x->next=front->next, front=x

C. rear->next=x, rear=x

D. rear->next=x, x->next=NULL, rear=x

17. 假设循环单链表表示的队列长度为 n，队头固定在链表尾，若只设头指针，则入队操作的时间复杂度为（　）。

A. $O(n)$　　B. $O(1)$　　C. $O(n^2)$　　D. $O(n\log_2 n)$

18. 假设输入序列为 1, 2, 3, 4, 5，利用两个队列进行出入队操作，不可能输出的序列是（　）。

A. 1, 2, 3, 4, 5　　B. 5, 2, 3, 4, 1　　C. 1, 3, 2, 4, 5　　D. 4, 1, 5, 2, 3

19. 若以 1, 2, 3, 4 作为双端队列的输入序列，则既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的输出序列是（　）。

A. 1, 2, 3, 4　　B. 4, 1, 3, 2　　C. 4, 2, 3, 1　　D. 4, 2, 1, 3

20. 【2010 统考真题】某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作，则不可能得到的出队序列是（　）。

A. b, a, c, d, e　　B. d, b, a, c, e　　C. d, b, c, a, e　　D. e, c, b, a, d

21. 【2011 统考真题】已知循环队列存储在一维数组 A[0…n-1]中，且队列非空时 front 和 rear 分别指向队首元素和队尾元素。若初始时队列为空，且要求第一个进入队列的元素存储在 A[0]处，则初始时 front 和 rear 的值分别是（　）。

A. 0，0　　B. 0，$n-1$　　C. $n-1$，0　　D. $n-1$，$n-1$

22. 【2014 统考真题】循环队列放在一维数组 A[0…M-1]中，end1 指向队首元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M-1 个元素。初始时为空。下列判断队空和队满的条件中，正确的是（　）。

A. 队空：end1==end2;　　队满：end1==(end2+1) mod M

B. 队空：end1==end2;　　队满：end2==(end1+1) mod (M-1)

C. 队空：end2==(end1+1)mod M;　　队满：end1==(end2+1) mod M

D. 队空：end1==(end2+1)mod M;　　队满：end2==(end1+1) mod (M-1)

23. 【2018 统考真题】现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：①出队并输出出队元素；②出队并将出队元素入栈；③出栈并输出出栈元素，则不能得到的输出序列是（　）。

A. 1, 2, 5, 6, 4, 3　　B. 2, 3, 4, 5, 6, 1　　C. 3, 4, 5, 6, 1, 2　　D. 6, 5, 4, 3, 2, 1

配套课程公众号[考机研]
永久联系微信:4550060

24.【2021统考真题】初始为空的队列Q的一端仅能进行入队操作，另外一端既能进行入队操作又能进行出队操作。若Q的入队序列是1, 2, 3, 4, 5，则不能得到的出队序列是(　)。

A. 5, 4, 3, 1, 2　　B. 5, 3, 1, 2, 4　　C. 4, 2, 1, 3, 5　　D. 4, 1, 3, 2, 5

二、综合应用题

01. 若希望循环队列中的元素都能得到利用，则需设置一个标志域tag，并以tag的值为0或1来区分队首指针front和队尾指针rear相同时的队列状态是“空”还是“满”。试编写与此结构相应的入队和出队算法。

02. Q是一个队列，S是一个空栈，实现将队列中的元素逆置的算法。

03. 利用两个栈S1和S2来模拟一个队列，已知栈的4个运算定义如下：

```
Push(S,x);                  //元素x入栈S
Pop(S,x);                   //S出栈并将出栈的值赋给x
StackEmpty(S);              //判断栈是否为空
StackOverflow(S);           //判断栈是否为满
```

如何利用栈的运算来实现该队列的3个运算（形参由读者根据要求自己设计）？

```
Enqueue;                    //将元素x入队
Dequeue;                    //出队，并将出队元素存储在x中
QueueEmpty;                 //判断队列是否为空
```

04.【2019统考真题】请设计一个队列，要求满足：①初始时队列为空；②入队时，允许增加队列占用空间；③出队后，出队元素所占用的空间可重复使用，即整个队列所占用的空间只增不减；④入队操作和出队操作的时间复杂度始终保持为$O(1)$。请回答：

1）该队列是应选择链式存储结构，还是应选择顺序存储结构？

2）画出队列的初始状态，并给出判断队空和队满的条件。

3）画出第一个元素入队后的队列状态。

4）给出入队操作和出队操作的基本过程。

3.2.6　答案与解析

一、单项选择题

01. D

栈和队列的逻辑结构都是线性结构，都可以采用顺序存储或链式存储，选项C显然也错误。只有选项D才是栈和队列的本质区别，限定表中插入和删除操作位置的不同。

02. B

队列“先进先出”的特性表现在：先入队列的元素先出队列，后入队列的元素后出队列，入队对应的是插入操作，出队对应的是删除操作。选项I和IV均正确。

03. D

删除队首元素即出队，是队列的基本操作之一，所以选择选项D。

04. B

队列的入队顺序和出队顺序是一致的，这是和栈不同的。

05. D

数组下标范围为0～n，因此数组容量为n+1。循环队列中元素入队的操作是rear=(rear+1) mod maxsize，题中maxsize=n+1。因此入队操作应为rear=(rear+1) mod (n+1)。

06. C

队列的长度为(rear-front+maxsize)%maxsize=(rear-front+21)%21=16。这种情

配套课程公众号[考机研]
永久联系微信:4550060

况和 front 指向当前元素，rear 指向队尾元素的下一个元素的计算是相同的。

注　意

数组 A[n]的下标范围为 0～n-1。若写成 A[0…n]，则说明下标范围为 0～n。

07．B

循环队列中，每删除一个元素，队首指针 front=(front+1)%6，每插入一个元素，队尾指针 rear=(rear+1)%6。上述操作后，front=0，rear=3。

08．D

当队列中只有一个元素时，front 指向该元素的前一个位置，rear 指向该元素，因此当队列为空时，队首指针等于队尾指针，这样第一个元素入队后，才能符合题目要求。

09．C

既然不能附加任何其他数据成员，只能采用牺牲一个存储单元的方法来区分是队空还是队满，约定以“队列头指针在队尾指针的下一位置作为队满的标志”，因此选择选项 C。选项 A 是判断队列是否空的条件，选项 B 和 D 都是干扰项。

注　意

对于这类具体问题，举一些特例判断往往比直接思考问题能更快得到答案。

10．A

若用(rear+1)%(n+1)==front 作为队满标志，则说明题目采用了牺牲一个存储单元的方法来区分队空和队满，因此可用 front==rear 作为队空标志。

11．D

虽然链式队列采用动态分配方式，但其长度也受内存空间的限制，不能无限制增长。顺序队列和链式队列的入队和出队时间复杂度均为 $O(1)$。顺序队列和链式队列都可以进行顺序访问。对于顺序队列，可通过队首指针和队尾指针计算队列中的元素个数，而链式队列则不能。

12．B

因为队列需在双端进行操作，所以选项 C 和 D 的链表显然不太适合链队。对于选项 A，链表在完成入队和出队后还要修改为循环的，对于队列来讲这是多余的（画蛇添足）。对于选项 B，因为有首指针，所以适合删除首结点；因为有队尾指针，所以适合在其后插入结点。

13．A

因为非循环双链表只带队首指针，所以在执行入队操作时需要修改队尾结点的指针域，而查找队尾结点需要 $O(n)$的时间。选项 B、C 和 D 均可在 $O(1)$的时间内找到队首和队尾。

14．A

因为在队头做出队操作，为便于删除队首元素，所以总是选择链头作为队头。

15．D

队列用链式存储时，删除元素从表头删除，通常仅需修改头指针，但若队列中仅有一个元素，则队尾指针也需要被修改，当仅有一个元素时，删除后队列为空，需修改队尾指针为 rear=front。

16．D

插入操作时，先将结点 x 插入到链表尾部，再让 rear 指向这个结点 x。选项 C 的做法不够严密，因为是队尾，所以队尾 x->next 必须置为空。

王道计算机教育 www.wangdaokaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

17. A

依题意，入队操作是在队尾进行，即链表表头。题中已明确说明链表只设头指针，即没有头结点和尾指针，入队后，循环单链表必须保持循环的性质，在只带头指针的循环单链表中寻找表尾结点的时间复杂度为$O(n)$，所以入队的时间复杂度为$O(n)$。

18. B

此类题可对各选项进行模拟，假设队列为Q_1和Q_2。对于选项A，元素依次入队Q_1，然后依次出队。对于选项B，5最先出队，只可能是1, 2, 3, 4入队Q_1，5入队Q_2，然后5出队，只能得到5, 1, 2, 3, 4。对于选项C，1, 2, 5入队Q_1，3, 4入队Q_2，然后按要求出队。选项D的分析同选项C。

19. C

使用排除法。先看可由输入受限的双端队列产生的序列：设右端输入受限，1, 2, 3, 4依次左入，则依次左出可得4, 3, 2, 1，排除选项A；左出、右出、左出、左出可得到4, 1, 3, 2，排除选项B；再看可由输出受限的双端队列产生的序列：设右端输出受限，1, 2, 3, 4依次左入、左入、右入、左入，依次左出可得到4, 2, 1, 3，排除选项D。

20. C

本题的队列实际上是一个输出受限的双端队列，如图3.11所示。A操作：a左入（或右入）、b左入、c右入、d右入、e右入。B操作：a左入（或右入）、b左入、c右入、d左入、e右入。D操作：a左入（或右入）、b左入、c左入、d右入、e左入。C操作：a左入（或右入）、b右入、因d未出，此时只能入队，c怎么进都不可能在b和a之间。

【另解】初始时队列为空，第1个元素a左入（或右入）后，第2个元素b无论是左入还是右入都必与a相邻，而选项C中a与b不相邻，不合题意。

21. B

根据题意，第一个元素进入队列后存储在`A[0]`处，此时`front`和`rear`值都为`0`。入队时因为要执行`(rear+1)%n`操作，所以若入队后指针指向`0`，则`rear`初值为`n-1`，而因为第一个元素在`A[0]`中，插入操作只改变`rear`指针，所以`front`为`0`不变。

注　意

①循环队列是指顺序存储的队列，而不是指逻辑上的循环，如循环单链表表示的队列不能称为循环队列。②front和rear的初值并不是固定的。

22. A

`end1`指向队首元素，可知出队操作是先从`A[end1]`读数，然后`end1`再加`1`。`end2`指向队尾元素的后一个位置，可知入队操作是先存数到`A[end2]`，然后`end2`再加`1`。若用`A[0]`存储第一个元素，则队列初始时，入队操作先把数据放到`A[0]`中，然后`end2`自增，即可知`end2`初值为`0`；而`end1`指向的是队首元素，队首元素在数组`A`中的下标为`0`，所以得知`end1`的初值也为`0`，可知队空条件为`end1==end2`；然后考虑队列满时，因为队列最多能容纳`M-1`个元素，假设队列存储在下标为`0`到`M-2`的`M-1`个区域，队头为`A[0]`，队尾为`A[M-2]`，此时队列满，考虑在这种情况下`end1`和`end2`的状态，`end1`指向队首元素，可知`end1=0`，`end2`指向队尾元素的后一个位置，可知`end2=M-2+1=M-1`，所以队满条件为`end1==(end2+1) mod M`。

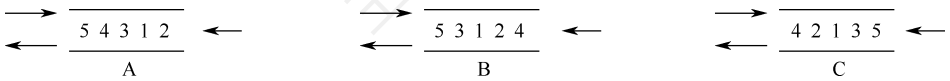
23. C

选项A的操作顺序为①①②②①①③③。选项B的操作顺序为②①①①①①③。选项D的操作顺序为②②②②②①③③③③③。对于选项C：首先输出3，说明1和2必须先依次入栈，而此后2肯定比1先输出，因此无法得到1, 2的输出顺序。

配套课程公众号[考机研]
永久联系微信:4550060

24. D

假设队列左端允许入队和出队，右端只能入队。对于选项 A，依次从右端入队 1, 2，再从左端入队 3, 4, 5。对于选项 B，从右端入队 1, 2，然后从左端入队 3，再从右端入队 4，最后从左端入队 5。对于选项 C，从左端入队 1, 2，然后从右端入队 3，再从左端入队 4，最后从右端入队 5。无法验证选项 D 的序列。



二、综合应用题

01.【解答】

在循环队列的类型结构中，增设一个整型变量 tag，入队时置 tag 为 1，出队时置 tag 为 0（因为只有入队操作可能导致队满，也只有出队操作可能导致队空）。队列 Q 初始时，置 tag=0、front=rear=0。这样队列的 4 要素如下：

队空条件：Q.front==Q.rear 且 Q.tag==0。

队满条件：Q.front==Q.rear 且 Q.tag==1。

入队操作：Q.data[Q.rear]=x; Q.rear=(Q.rear+1)%MaxSize; Q.tag=1。

出队操作：x=Q.data[Q.front]; Q.front=(Q.front+1)%MaxSize; Q.tag=0。

1）设“tag”法的循环队列入队算法：

```
int EnQueue1(SqQueue &Q,ElemType x){
    if(Q.front==Q.rear&&Q.tag==1)
return 0;                          //两个条件都满足时则队满
    Q.data[Q.rear]=x;
    Q.rear=(Q.rear+1)%MaxSize;
    Q.tag=1;                       //可能队满
    return 1;
}
```

2）设“tag”法的循环队列出队算法：

```
int DeQueue1(SqQueue &Q,ElemType &x){
    if(Q.front==Q.rear&&Q.tag==0)
        return 0;                  //两个条件都满足时则队空
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;
    Q.tag=0;                       //可能队空
    return 1;
}
```

02.【解答】

本题主要考查大家对队列和栈的特性与操作的理解。因为对队列的一系列操作不可能将其中的元素逆置，而栈可以将入栈的元素逆序提取出来，所以我们可以让队列中的元素逐个地出队，入栈；全部入栈后再逐个出栈，入队。

算法的实现如下：

```
void Inverser(Stack &S,Queue &Q){
//本算法实现将队列中的元素逆置
    while(!QueueEmpty(Q)){
        x=DeQueue(Q);              //队列中全部元素依次出队
        Push(S,x);                 //元素依次入栈
    }
    while(!StackEmpty(S)){
        Pop(S,x);                  //栈中全部元素依次出栈
```

配套课程公众号[考机研]
永久联系微信:4550060

```
            EnQueue(Q,x);                    //再入队
        }
    }
```

03.【解答】

利用两个栈 S1 和 S2 来模拟一个队列，当需要向队列中插入一个元素时，用 S1 来存放已输入的元素，即 S1 执行入栈操作。当需要出队时，则对 S2 执行出栈操作。因为从栈中取出元素的顺序是原顺序的逆序，所以必须先将 S1 中的所有元素全部出栈并入栈到 S2 中，再在 S2 中执行出栈操作，即可实现出队操作，而在执行此操作前必须判断 S2 是否为空，否则会导致顺序混乱。当栈 S1 和 S2 都为空时队列为空。

总结如下：

1）对 S2 的出栈操作用作出队，若 S2 为空，则先将 S1 中的所有元素送入 S2。

2）对 S1 的入栈操作用作入队，若 S1 满，必须先保证 S2 为空，才能将 S1 中的元素全部插入 S2 中。

入队算法：

```
int EnQueue(Stack &S1,Stack &S2,ElemType e){
    if(!StackOverflow(S1)){
        Push(S1,e);
        return 1;
    }
    if(StackOverflow(S1)&&!StackEmpty(S2)){
        printf("队列为满");
        return 0;
    }
    if(StackOverflow(S1)&&StackEmpty(S2)){
        while(!StackEmpty(S1)){
            Pop(S1,x);
            Push(S2,x);
        }
    }
    Push(S1,e);
    return 1;
}
```

出队算法：

```
void DeQueue(Stack &S1,Stack &S2,ElemType &x){
    if(!StackEmpty(S2)){
        Pop(S2,x);
    }
    else if(StackEmpty(S1)){
        printf("队列为空");
    }
    else{
        while(!StackEmpty(S1)){
            Pop(S1,x);
            Push(S2,x);
        }
    Pop(S2,x);
    }
}
```

判断队列为空的算法：

```
int QueueEmpty(Stack S1,Stack S2){
    if(StackEmpty(S1)&&StackEmpty(S2))
        return 1;
    else
        return 0;
}
```

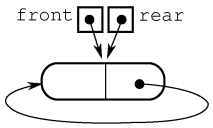
王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

04.【解答】

1）顺序存储无法满足要求②的队列占用空间随着入队操作而增加。根据要求来分析：要求①容易满足；链式存储方便开辟新空间，要求②容易满足；对于要求③，出队后的结点并不真正释放，用队首指针指向新的队头结点，新元素入队时，有空余结点则无须开辟新空间，赋值到队尾后的第一个空结点即可，然后用队尾指针指向新的队尾结点，这就需要设计成一个首尾相接的循环单链表，类似于循环队列的思想。设置队头、队尾指针后，链式队列的入队操作和出队操作的时间复杂度均为 $O(1)$，要求④可以满足。

因此，采用链式存储结构（两段式单向循环链表），队首指针为 front，队尾指针为 rear。

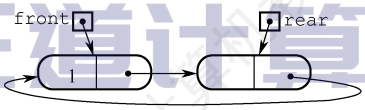
2）该循环链式队列的实现可以参考循环队列，不同之处在于循环链式队列可以方便地增加空间，出队的结点可以循环利用，入队时空间不够也可以动态增加。同样，循环链式队列也要区分队满和队空的情况，这里参考循环队列牺牲一个单元来判断。初始时，创建只有一个空结点的循环单链表，头指针 front 和尾指针 rear 均指向空结点，如下图所示。



队空的判定条件：front==rear。

队满的判定条件：front==rear->next。

3）插入第一个元素后的状态如下图所示。



4）操作的基本过程如下：

入队操作：
若（front==rear->next）　　//队满 　　则在rear后面插入一个新的空闲结点； 入队元素保存到rear所指结点中；rear=rear->next；返回。
出队操作：
若（front==rear）　　//队空 　　则出队失败，返回； 取front所指结点中的元素e；front=front->next；返回e。

3.3　栈和队列的应用

要熟练掌握栈和队列，必须学习栈和队列的应用，把握其中的规律，然后举一反三。接下来将简单介绍栈和队列的一些常见应用。

3.3.1　栈在括号匹配中的应用

假设表达式中允许包含两种括号：圆括号和方括号，其嵌套的顺序任意即（[]()）或[([][])]等均为正确的格式，[(])或([()]或(()]均为不正确的格式。

考虑下列括号序列：

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

[	(	[	]	[	]	)	]
1	2	3	4	5	6	7	8

分析如下：

1）计算机接收第1个括号“[”后，期待与之匹配的第8个括号“]”出现。

2）获得了第2个括号“(”，此时第1个括号“[”暂时放在一边，而急迫期待与之匹配的第7个括号“)”出现。

3）获得了第3个括号“[”，此时第2个括号“(”暂时放在一边，而急迫期待与之匹配的第4个括号“]”出现。第3个括号的期待得到满足，消解之后，第2个括号的期待匹配又成为当前最急迫的任务。

4）以此类推，可见该处理过程与栈的思想吻合。

算法的思想如下：

1）初始设置一个空栈，顺序读入括号。

2）若是左括号，则作为一个新的更急迫的期待压入栈中，自然使原有的栈中所有未消解的期待的急迫性降了一级。

3）若是右括号，则或使置于栈顶的最急迫期待得以消解，或是不合法的情况（括号序列不匹配，退出程序）。算法结束时，栈为空，否则括号序列不匹配。

3.3.2 栈在表达式求值中的应用

表达式求值是程序设计语言编译中一个最基本的问题，它是栈应用的一个典型范例。

1．算术表达式

中缀表达式（如3+4）是人们常用的算术表达式，操作符以中缀形式处于操作数的中间。与前缀表达式（如+34）或后缀表达式（如34+）相比，中缀表达式不容易被计算机解析，但仍被许多程序语言使用，因为它更符合人们的思维习惯。

与前缀表达式或后缀表达式不同的是，中缀表达式中的括号是必需的。计算过程中必须用括号将操作符和对应的操作数括起来，用于指示运算的次序。后缀表达式的运算符在操作数后面，后缀表达式中考虑了运算符的优先级，没有括号，只有操作数和运算符。

中缀表达式A+B*(C-D)-E/F对应的后缀表达式为ABCD-*+EF/-，将后缀表达式与原表达式对应的表达式树（图3.15）的后序遍历序列进行比较，可发现它们有异曲同工之妙。

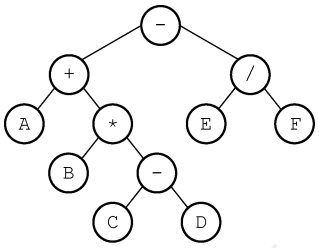


图3.15 A+B*(C-D)-E/F对应的表达式树

2．中缀表达式转后缀表达式

命题追踪 ▶▶ 中缀表达式转后缀表达式的方法（2024）

下面先给出一种由中缀表达式转后缀表达式[①]的手算方法。

① 中缀转前缀的方法类似，且统考真题仅考查过中缀转后缀的过程，因此本节仅介绍中缀转后缀的方法。

配套课程公众号[考机研]
永久联系微信:4550060

1）按照运算符的运算顺序对所有运算单位加括号。

2）将运算符移至对应括号的后面，相当于按“左操作数 右操作数 运算符”重新组合。

3）去除所有括号。

例如，中缀表达式 A+B*(C-D)-E/F 转后缀表达的过程如下（下标表示运算符的运算顺序）：

1）加括号：((A+③(B*②(C-①D)))-⑤(E/④F))。

2）运算符后移：((A(B(CD)-①)*②)+③(EF)/④)-⑤。

3）去除括号后，得到后缀表达式：ABCD-①*②+③EF/④-⑤。

在计算机中，中缀表达式转后缀表达式时需要借助一个栈，用于保存暂时还不能确定运算顺序的运算符。从左到右依次扫描中缀表达式中的每一项，具体转化过程如下：

1）遇到操作数。直接加入后缀表达式。

2）遇到界限符。若为“(”，则直接入栈；若为“)”，则不入栈，且依次弹出栈中的运算符并加入后缀表达式，直到遇到“(”为止，并直接删除“(”。

3）遇到运算符。① 若其优先级高于栈顶运算符或遇到栈顶为“(”，则直接入栈；② 若其优先级低于或等于栈顶运算符，则依次弹出栈中的运算符并加入后缀表达式，直到遇到一个优先级低于它的运算符或遇到“(”或栈空为止，之后将当前运算符入栈。

按上述方法扫描所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

例如，中缀表达式 A+B*(C-D)-E/F 转后缀表达式的过程如表 3.1 所示。

表 3.1　中缀表达式 A+B*(C-D)-E/F 转后缀表达式的过程

步	待处理序列	栈内	后缀表达式	扫描项	说　明
1	A+B*(C-D)-E/F			A	A 加入后缀表达式
2	+B*(C-D)-E/F		A	+	+入栈
3	B*(C-D)-E/F	+	A	B	B 加入后缀表达式
4	*(C-D)-E/F	+	AB	*	*优先级高于栈顶，*入栈
5	(C-D)-E/F	+*	AB	(	(直接入栈
6	C-D)-E/F	+*(	AB	C	C 加入后缀表达式
7	-D)-E/F	+*(	ABC	-	栈顶为(，-直接入栈
8	D)-E/F	+*(-	ABC	D	D 加入后缀表达式
9	)-E/F	+*(-	ABCD	)	遇到)，弹出-，删除(
10	-E/F	+*	ABCD-	-	-优先级低于栈顶，依次弹出*、+，-入栈
11	E/F	-	ABCD-*+	E	E 加入后缀表达式
12	/F	-	ABCD-*+E	/	/优先级高于栈顶，/入栈
13	F	-/	ABCD-*+E	F	F 加入后缀表达式
14		-/	ABCD-*+EF		字符扫描完毕，弹出剩余运算符
			ABCD-*+EF/-		结束

命题追踪 ▶▶ 栈的深度分析（2009、2012）

所谓栈的深度，是指栈中的元素个数，通常是给出入栈和出栈序列，求最大深度（栈的容量应大于或等于最大深度）。有时会间接给出入栈和出栈序列，例如以中缀表达式和后缀表达式的形式给出入栈和出栈序列。掌握栈的先进后出的特点进行手工模拟是解决这类问题的有效方法。

3．后缀表达式求值

命题追踪 ▶▶ 用栈实现表达式求值的分析（2018）

通过后缀表示计算表达式值的过程：从左往右依次扫描表达式的每一项，若该项是操作数，

配套课程公众号[考机研]
永久联系微信:4550060

则将其压入栈中；若该项是操作符<op>，则从栈中退出两个操作数Y和X，形成运算指令X<op>Y，并将计算结果压入栈中。当所有项都扫描并处理完后，栈顶存放的就是最后的计算结果。

例如，后缀表达式ABCD-*+EF/-求值的过程需要12步，见表3.2。

表3.2 后缀表达式ABCD-*+EF/-求值的过程

步	扫描项	项类型	动作	栈中内容
1			置空栈	空
2	A	操作数	入栈	A
3	B	操作数	入栈	A B
4	C	操作数	入栈	A B C
5	D	操作数	入栈	A B C D
6	-	操作符	D、C出栈，计算C-D，结果R_1入栈	A B R_1
7	*	操作符	R_1、B出栈，计算B×R_1，结果R_2入栈	A R_2
8	+	操作符	R_2、A出栈，计算A+R_2，结果R_3入栈	R_3
9	E	操作数	入栈	R_3 E
10	F	操作数	入栈	R_3 E F
11	/	操作符	F、E出栈，计算E/F，结果R_4入栈	R_3 R_4
12	-	操作符	R_4、R_3出栈，计算R_3-R_4，结果R_5入栈	R_5

3.3.3 栈在递归中的应用

递归是一种重要的程序设计方法。简单地说，若在一个函数、过程或数据结构的定义中又应用了它自身，则这个函数、过程或数据结构称为是递归定义的，简称递归。

递归通常把一个大型的复杂问题层层转化为一个与原问题相似的规模较小的问题来求解，递归策略只需少量的代码就可以描述出解题过程所需要的多次重复计算，大大减少了程序的代码量。但在通常情况下，它的效率并不是太高。

以斐波那契数列为例，其定义为

$$F(n)=\begin{cases}F(n-1)+F(n-2), & n>1\\ 1, & n=1\\ 0, & n=0\end{cases}$$

这就是递归的一个典型例子，用程序实现时如下：

```
int F(int n){                                    //斐波那契数列的实现
    if(n==0)
        return 0;                                //边界条件
    else if(n==1)
        return 1;                                //边界条件
    else
        return F(n-1)+F(n-2);                    //递归表达式
}
```

必须注意递归模型不能是循环定义的，其必须满足下面的两个条件：

- 递归表达式（递归体）。
- 边界条件（递归出口）。

递归的精髓在于能否将原始问题转换为属性相同但规模较小的问题。

命题追踪 ▶▶ 栈在函数调用中的作用和工作原理（2015、2017）

在递归调用的过程中，系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来进行数据存储，递归次数过多容易造成栈溢出等。而其效率不高的原因是递归调用过程中包含很

王道计算机教育 www.kaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

多重复的计算。下面以 $n=5$ 为例，列出递归调用执行过程，如图 3.16 所示。

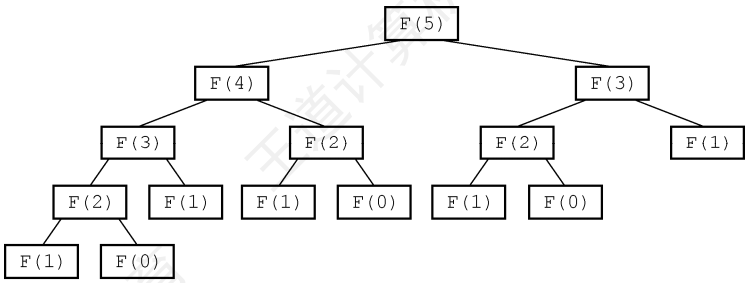


图 3.16　F(5)的递归执行过程

显然，在递归调用的过程中，F(3)被计算 2 次，F(2)被计算 3 次。F(1)被调用 5 次，F(0)被调用 3 次。所以，递归的效率低下，但优点是代码简单，容易理解。在第 5 章的树中利用了递归的思想，代码变得十分简单。通常情况下，初学者很难理解递归的调用过程，若读者想具体了解递归是如何实现的，可以参阅编译原理教材中的相关内容。

可以将递归算法转换为非递归算法，通常需要借助栈来实现这种转换。

3.3.4　队列在层次遍历中的应用

在信息处理中有一大类问题需要逐层或逐行处理。这类问题的解决方法往往是在处理当前层或当前行时就对下一层或下一行做预处理，把处理顺序安排好，等到当前层或当前行处理完毕，就可以处理下一层或下一行。使用队列是为了保存下一步的处理顺序。下面用二叉树（见图 3.17）层次遍历的例子，说明队列的应用。表 3.3 显示了层次遍历二叉树的过程。

王道计算机教育 www.cskaoyan.com

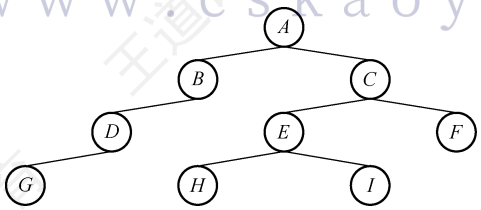


图 3.17　二叉树

表 3.3　层次遍历二叉树的过程

序	说　明	队内	队　外
1	*A* 入	*A*	
2	*A* 出，*BC* 入	*BC*	*A*
3	*B* 出，*D* 入	*CD*	*AB*
4	*C* 出，*EF* 入	*DEF*	*ABC*
5	*D* 出，*G* 入	*EFG*	*ABCD*
6	*E* 出，*HI* 入	*FGHI*	*ABCDE*
7	*F* 出	*GHI*	*ABCDEF*
8	*GHI* 出		*ABCDEFGHI*

该过程的简单描述如下：

① 根结点入队。

② 若队空（所有结点都已处理完毕），则结束遍历；否则重复③操作。

配套课程公众号[考机研]
永久联系微信:4550060

③ 队列中第一个结点出队，并访问之。若其有左孩子，则将左孩子入队；若其有右孩子，则将右孩子入队，返回②。

3.3.5 队列在计算机系统中的应用

队列在计算机系统中的应用非常广泛，以下仅从两个方面来阐述：第一个方面是解决主机与外部设备之间速度不匹配的问题，第二个方面是解决由多用户引起的资源竞争问题。

命题追踪 ▶▶ 缓冲区的逻辑结构（2009）

对于第一个方面，仅以主机和打印机之间速度不匹配的问题为例做简要说明。主机输出数据给打印机打印，输出数据的速度比打印数据的速度要快得多，因为速度不匹配，若直接把输出的数据送给打印机打印，则显然是不行的。解决的方法是设置一个打印数据缓冲区，主机把要打印输出的数据依次写入这个缓冲区，写满后就暂停输出，转去做其他的事情。打印机就从缓冲区中按照先进先出的原则依次取出数据并打印，打印完后再向主机发出请求。主机接到请求后再向缓冲区写入打印数据。这样做既保证了打印数据的正确，又使主机提高了效率。由此可见，打印数据缓冲区中所存储的数据就是一个队列。

命题追踪 ▶▶ 多队列出队/入队操作的应用（2016）

对于第二个方面，CPU（中央处理器，它包括运算器和控制器）资源的竞争就是一个典型的例子。在一个带有多终端的计算机系统上，有多个用户需要CPU各自运行自己的程序，它们分别通过各自的终端向操作系统提出占用CPU的请求。操作系统通常按照每个请求在时间上的先后顺序，把它们排成一个队列，每次把CPU分配给队首请求的用户使用。当相应的程序运行结束或用完规定的时间间隔后，令其出队，再把CPU分配给新的队首请求的用户使用。这样既能满足每个用户的请求，又使CPU能够正常运行。

王道计算机教育 www.cskaoyan.com

3.3.6 本节试题精选

一、单项选择题

01. 栈的应用不包括（ ）。

A. 递归　　B. 表达式求值　　C. 括号匹配　　D. 缓冲区

02. 表达式a*(b+c)-d的后缀表达式是（ ）。

A. abcd*+-　　B. abc+*d-　　C. abc*+d-　　D. -+*abcd

03. 下面（ ）用到了队列。

A. 括号匹配　　B. 表达式求值　　C. 递归　　D. FIFO页面替换算法

04. 利用栈求表达式的值时，设立运算数栈OPEN。假设OPEN只有两个存储单元，则在下列表达式中，不会发生溢出的是（ ）。

A. A-B*(C-D)　　B. (A-B)*C-D　　C. (A-B*C)-D　　D. (A-B)*(C-D)

05. 执行完下列语句段后，i的值为（ ）。

```
int f(int x){
    return ((x>0)? x*f(x-1):2);
}
int i;
i=f(f(1));
```

A. 2　　B. 4　　C. 8　　D. 无限递归

配套课程公众号[考机研]
永久联系微信:4550060

06. 设有如下递归函数，则计算 F(8)需要调用该递归函数的次数为（　）。

```
int F(int n){
    if(n<=3) return 1;
    else return F(n-2)+F(n-4)+1;
}
```

A. 7　　B. 8　　C. 9　　D. 10

07. 设有如下递归函数，在 func(func(5))的执行过程中，第 4 个被执行的 func 函数是（　）。

```
int func(int x){
    if(x<=3) return 2;
    else return func(x-2)+func(x-4);
}
```

A. func(2)　　B. func(3)　　C. func(4)　　D. func(5)

08. 对于一个问题的递归算法求解和其相对应的非递归算法求解，（　）。

A. 递归算法通常效率高一些　　B. 非递归算法通常效率高一些

C. 两者相同　　D. 无法比较

09. 执行函数时，其局部变量一般采用（　）进行存储。

A. 树形结构　　B. 静态链表　　C. 栈结构　　D. 队列结构

10. 执行（　）操作时，需要使用队列作为辅助存储空间。

A. 查找散列（哈希）表　　B. 广度优先搜索图

C. 前序（根）遍历二叉树　　D. 深度优先搜索图

11. 下列说法中，正确的是（　）。

A. 消除递归不一定需要使用栈

B. 对同一输入序列进行两组不同的合法入栈和出栈组合操作，所得的输出序列也一定相同

C. 通常使用队列来处理函数或过程调用

D. 队列和栈都是运算受限的线性表，只允许在表的两端进行运算

12. 【2009 统考真题】为解决计算机主机与打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是（　）。

A. 栈　　B. 队列　　C. 树　　D. 图

13. 【2012 统考真题】已知操作符包括“+”“-”“*”“/”“(”和“)”。将中缀表达式 a+b-a*((c+d)/e-f)+g 转换为等价的后缀表达式 ab+acd+e/f-*-g+时，用栈来存放暂时还不能确定运算次序的操作符。栈初始时为空时，转换过程中同时保存在栈中的操作符的最大个数是（　）。

A. 5　　B. 7　　C. 8　　D. 11

14. 【2014 统考真题】假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是（　）。

A. +(*-　　B. +(-*　　C. /+(*-*　　D. /+-*

15. 【2015 统考真题】已知程序如下：

```
int S(int n)
{     return (n<=0)?0:S(n-1)+n;}
void main()
{     cout<<S(1);}
```

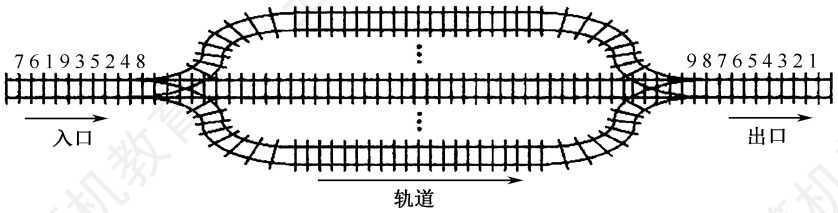
程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是（　）。

配套课程公众号[考机研]
永久联系微信:4550060

A. main()→S(1)→S(0)　　B. S(0)→S(1)→main()

C. main()→S(0)→S(1)　　D. S(1)→S(0)→main()

16. 【2016统考真题】设有如下图所示的火车车轨，入口和出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为1~9的9列列车，驶入的次序依次是8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为1~9，则 n 至少是（　）。



A. 2　　B. 3　　C. 4　　D. 5

17. 【2017统考真题】下列关于栈的叙述中，错误的是（　）。

I. 采用非递归方式重写递归程序时必须使用栈

II. 函数调用时，系统要用栈保存必要的信息

III. 只要确定了入栈次序，即可确定出栈次序

IV. 栈是一种受限的线性表，允许在其两端进行操作

A. 仅I　　B. 仅I、II、III　　C. 仅I、III、IV　　D. 仅II、III、IV

18. 【2018统考真题】若栈S1中保存整数，栈S2中保存运算符，函数F()依次执行下述各步操作：

1）从S1中依次弹出两个操作数a和b。

2）从S2中弹出一个运算符op。

3）执行相应的运算b op a。

4）将运算结果压入S1中。

假定S1中的操作数依次是5, 8, 3, 2（2在栈顶），S2中的运算符依次是*、-、+（+在栈顶）。调用3次F()后，S1栈顶保存的值是（　）。

A. −15　　B. 15　　C. −20　　D. 20

19. 【2024统考真题】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是（　）。

A. $xyzu-*v/+$　　B. $xyzu-v/*+$　　C. $+x/*y-zuv$　　D. $+x*y/-zuv$

二、综合应用题

01. 假设一个算术表达式中包含圆括号、方括号和花括号3种类型的括号，编写一个算法来判别表达式中的括号是否配对，以字符“\0”作为算术表达式的结束符。

3.3.7 答案与解析

一、单项选择题

01. D

缓冲区是用队列实现的，选项A、B、C都是栈的典型应用。

02. B

后缀表达式中，每个运算符均直接位于其两个操作数的后面，按照这样的方式逐步根据计算的优先级将每个计算式进行变换，即可得到后缀表达式。

【另解】将两个直接操作数用括号括起来，再将操作符提到括号后，最后去掉括号。例如，

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

对于(①(②a*(③b+c))-d)，提出操作符并去掉括号后，可得后缀表达式为 abc+*d-。

学完第 5 章后，可将表达式画成二叉树的形式，再用后序遍历即可求得后缀表达式。

03．D

FIFO 页面替换算法用到了队列。其余的都只用到了栈。

04．B

利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得栈深为 2。选项 A、C、D 依次计算得栈深为 4、3、3。因此选择选项 B。

技　巧

根据运算符优先级，统计已依次入栈但还未参与计算的运算符的个数。以选项 C 为例，当“(”“A”“-”入栈时，“(”和“-”还未参与运算，此时运算符栈大小为 2，“B”和“*”入栈时运算符大小为 3，“C”入栈时“B*C”运算，运算符栈大小为 2，以此类推。

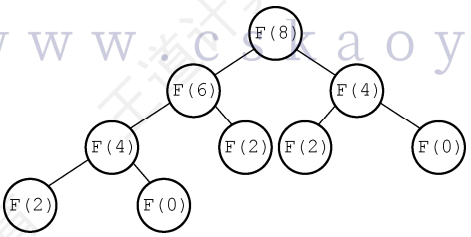
05．B

栈与递归有着紧密的联系。递归模型包括递归出口和递归体两个方面。递归出口是递归算法的出口，即终止递归的条件。递归体是一个递推的关系式。根据题意有

```
f(0)=2;
f(1)=1*f(0)=2;
f(f(1))=f(2)=2*f(1)=4。
```

06．C

计算 F(8)的递归调用树如下图所示：



由图可知，递归函数 F()调用的次数为 9。

07．C

首先，执行内层参数 func(5)=func(3)+func(1)=4，共执行 3 次 func 函数。然后，执行 func(func(5))=func(4)=func(2)+func(0)=4，因此第 4 个被执行的 func 函数是 func(4)。也可以采用画出递归调用树的方式，即某个函数的执行次序等于其在递归调用树的先序遍历中的次序。

08．B

通常情况下，递归算法在计算机实际执行的过程中包含很多的重复计算，所以效率会低。

09．C

调用函数时，系统会为调用者构造一个由参数表和返回地址组成的活动记录，并将记录压入系统提供的栈中，若被调用函数有局部变量，也要压入栈中。

10．B

本题涉及第 5 章和第 6 章的内容，图的广度优先搜索类似于树的层序遍历，都要借助于队列。

11．A

使用栈可以模拟递归的过程，以此来消除递归，但对于单向递归和尾递归而言，可以用迭代的方式来消除递归，选项 A 正确。不同的入栈和出栈组合操作，会产生许多不同的输出序列，B

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

错误。通常使用栈来处理函数或过程调用，选项C错误。队列和栈都是操作受限的线性表，但只有队列允许在表的两端进行运算，而栈只允许在栈顶方向进行操作，选项D错误。

12. B

在提取数据时必须保持原来数据的顺序，所以缓冲区的特性是先进先出。

13. A

在中缀表达式转后缀表达式的过程中，扫描到操作数时直接输出，扫描到操作符时根据其优先级进行相应的出入栈操作。有几点需要注意：①若遇到界限符“(”，则直接入栈；②若遇到界限符“)”，则不入栈，且依次弹出栈顶运算符，直到遇到“(”为止，并删除“(”；③若当前运算符的优先级高于栈顶运算符或遇到栈顶为“(”，则直接入栈；④若当前运算符的优先级低于或等于栈顶运算符，则依次弹出栈中的运算符并加入后缀表达式，直到遇到一个优先级低于它的运算符或遇到“(”或栈空为止，之后将当前运算符入栈。

求栈中操作符的最大个数时，为简单起见，可省略对操作数的处理。

步	待处理运算符	栈	扫描运算符	动　作
1	+b-a*((c+d)/e-f)+g		+	+入栈
2	-a*((c+d)/e-f)+g	+	-	-优先级等于栈顶+，弹出+，-入栈
3	*((c+d)/e-f)+g	-	*	*优先级高于栈顶-，*入栈
4	((c+d)/e-f)+g	-*	(	(直接入栈
5	(c+d)/e-f)+g	-*(	(	(直接入栈
6	+d)/e-f)+g	-*((	+	栈顶为(，+直接入栈
7	)/e-f)+g	-*((+	)	遇到)，弹出+，删除(
8	/e-f)+g	-*(	/	栈顶为(，/直接入栈
10	-f)+g	-*(/	-	-优先级低于栈顶/，弹出/，-入栈
11	)+g	-*(-	)	遇到)，弹出-，删除(
12	+g	-*	+	+优先级低于栈顶*，等于-，依次弹出*和-；+入栈
13		+		

由上述过程可知，栈中操作符的最大个数为5。

14. B

中缀表达式a/b+(c*d-e*f)/g转换为后缀表达式的过程如下：

步	待处理序列	栈	后缀表达式	扫描项	动　作
1	a/b+(c*d-e*f)/g			a	a加入后缀表达式
2	/b+(c*d-e*f)/g		a	/	/入栈
3	b+(c*d-e*f)/g	/	a	b	b加入后缀表达式
4	+(c*d-e*f)/g	/	ab	+	+优先级低于栈顶的/，弹出/，+入栈
5	(c*d-e*f)/g	+	ab/	(	(入栈
6	c*d-e*f)/g	+(	ab/	c	c加入后缀表达式
7	*d-e*f)/g	+(	ab/c	*	栈顶为(，*入栈
8	d-e*f)/g	+(*	ab/c	d	d加入后缀表达式
9	-e*f)/g	+(*	ab/cd	-	-优先级低于栈顶的*，弹出*，-入栈
10	e*f)/g	+(-	ab/cd*	e	e加入后缀表达式
11	*f)/g	+(-	ab/cd*e	*	*优先级高于栈顶的-，*入栈
12	f)/g	+(-*	ab/cd*e	f	f加入后缀表达式
13	)/g	+(-*	ab/cd*ef	)	遇到)，依次弹出*、-加入表达式，删除(
14	/g	+	ab/cd*ef*-	/	/优先级高于栈顶的+，/入栈
15	g	+/	ab/cd*ef*-	g	g加入后缀表达式
16		+/	ab/cd*ef*-g		扫描完毕，运算符依次弹出加入表达式
17			ab/cd*ef*-g/+		完成

配套课程公众号[考机研]
永久联系微信:4550060

由此可知，当扫描到 f 时，栈中的元素依次是+(-*。

【另解】采用手算方法，得出中缀式 a/b+(c*d-e*f)/g 对应的后缀式为 ab/cd*ef*-g/+。中缀表达式转后缀表达式时，操作数都直接输出，因此操作数的顺序是固定的。扫描到 f 时，在后缀表达式 f 后面的运算符要么还未入栈，要么还在栈中，需要结合中缀式来判断，f 后面依次出栈的运算符为*-/+，/在中缀表达式 f 之后，此时还未入栈，因此栈中的运算符（从栈底到栈顶）为+-*；此外，已入栈的界限符（此时还未消解，因此(也还在栈中，栈中的元素依次是+(-*。

15. A

递归调用函数时，在系统栈中保存的函数信息需满足先进后出的特点，依次调用了 main(),S(1),S(0)，所以栈底到栈顶的信息依次是 main(),S(1),S(0)。

注　意

在递归中，系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来存储。

16. C

根据题意：入队顺序为 8, 4, 2, 5, 3, 9, 1, 6, 7，出队顺序为 1～9。入口和出口之间有多个队列（n 条轨道），且每个队列（轨道）可容纳多个元素（多列列车），为便于区分，队列用字母编号。分析如下：显然先入队的元素必须小于后入队的元素（否则，若 8 和 4 入同一个队列，8 在 4 前面，则出队时也只能 8 在 4 前面），这样 8 入队列 A，4 入队列 B，2 入队列 C，5 入队列 B（按照前述原则“大的元素在小的元素后面”也可将 5 入队列 C，但这时剩下的元素 3 就必须放入一个新的队列中，无法确保“至少”），3 入队列 C，9 入队列 A，这时共占了 3 个队列，后面还有元素 1，直接再用一个新的队列 D，1 从队列 D 出队后，剩下的元素 6 和 7 或入队列 B，或入队列 C。综上，共占用了 4 个队列。当然还有其他的入队、出队情况，请读者自己推演，但要确保满足：①队列中后面的元素大于前面的元素；②确保占用最少（满足题意中“至少”）的队列。

17. C

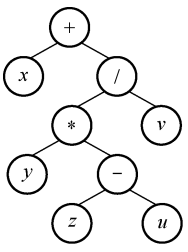
I 的反例：计算斐波那契数列迭代实现只需要一个循环即可实现。III 的反例：入栈序列为 1, 2，进行 Push, Push, Pop, Pop 操作，出栈次序为 2、1；进行 Push, Pop, Push, Pop 操作，出栈次序为 1, 2。IV，栈是一种受限的线性表，只允许在一端进行操作。II 正确。

18. B

第一次调用：①从 S1 中弹出 2 和 3；②从 S2 中弹出+；③执行 3+2=5；④将 5 压入 S1 中，第一次调用结束后 S1 中剩余 5、8、5（5 在栈顶），S2 中剩余*、-（-在栈顶）。第二次调用：①从 S1 中弹出 5 和 8；②从 S2 中弹出-；③执行 8-5=3；④将 3 压入 S1 中，第二次调用结束后 S1 中剩余 5、3（3 在栈顶），S2 中剩余*。第三次调用：①从 S1 中弹出 3 和 5；②从 S2 中弹出*；③执行 5*3=15；④将 15 压入 S1 中，第三次调用结束后 S1 中仅剩余 15，S2 为空。

19. A

根据中缀表达式画出对应的二叉树如右图所示，对该二叉树进行后序遍历即可得到后缀表达式为 $xyzu-*v/+$。本题也可采用本书前面描述的手算方法。



二、综合应用题

01.【解答】

括号匹配是栈的一个典型应用，给出这道题是希望读者好好掌握栈的应用。算法的基本思想是扫描每个字符，遇到花、方、圆的左括号时入栈，遇到花、方、圆的右括号时检查栈顶元素是否为相应的左括号，若是，出栈，否则配对错误。最后栈若不为空也为错误。

配套课程公众号[考机研]
永久联系微信:4550060

```
bool BracketsCheck(char *str){
    InitStack(S);              //初始化栈
    int i=0;
    while(str[i]!='\0'){
        switch(str[i]){
            //左括号入栈
            case '(': push(S,'('); break;
            case '[': push(S,'['); break;
            case '{': push(S,'{'); break;
            //遇到右括号，检测栈顶
            case ')': Pop(S,e);
                if(e!='(') return false;
            break;
            case ']': pop(S,e);
                if(e!='[') return false;
            break;
            case '}': pop(S,e);
                if(e!='{') return false;
            break;
            default:
            break;
        }//switch
        i++;
    }//while
    if(!IsEmpty(S)){
        printf("括号不匹配\n");
        return false;
    }
    else {
        printf("括号匹配\n");
        return true;
    }
}
```

王道计算机教育 www.cskaoyan.com

3.4 数组和特殊矩阵

矩阵在计算机图形学、工程计算中占有举足轻重的地位。在数据结构中考虑的是如何用最小的内存空间来存储同样的一组数据。所以，我们不研究矩阵及其运算等，而把精力放在如何将矩阵更有效地存储在内存中，并能方便地提取矩阵中的元素。

3.4.1 数组的定义

数组是由 n（$n \geq 1$）个相同类型的数据元素构成的有限序列，每个数据元素称为一个*数组元素*，每个元素在 n 个线性关系中的序号称为该元素的下标，下标的取值范围称为数组的维界。

数组与线性表的关系：数组是线性表的推广。一维数组可视为一个线性表；二维数组可视为其元素是定长数组的线性表，以此类推。数组一旦被定义，其维数和维界就不再改变。因此，除结构的初始化和销毁外，数组只会有存取元素和修改元素的操作。

3.4.2 数组的存储结构

大多数计算机语言都提供了数组数据类型，逻辑意义上的数组可采用计算机语言中的数组数据类型进行存储，一个数组的所有元素在内存中占用一段连续的存储空间。

配套课程公众号[考机研]
永久联系微信:4550060

以一维数组 A[0…n-1]为例，其存储结构关系式为

$$\mathrm{LOC}(a_i)=\mathrm{LOC}(a_0)+i\times L\quad(0\leqslant i<n)$$

其中，L 是每个数组元素所占的存储单元。

命题追踪 ▶▶ 二维数组按行优先存储的下标对应关系（2021）

对于多维数组，有两种映射方法：按行优先和按列优先。以二维数组为例，按行优先存储的基本思想是：先行后列，先存储行号较小的元素，行号相等先存储列号较小的元素。设二维数组的行下标与列下标的范围分别为$[0,h_1]$与$[0,h_2]$，则存储结构关系式为

$$\mathrm{LOC}(a_{i,j})=\mathrm{LOC}(a_{0,0})+[i\times(h_2+1)+j]\times L$$

例如，对于数组 $\boldsymbol{A}_{[2][3]}$，它按行优先方式在内存中的存储形式如图 3.18 所示。

$$A_{[2][3]}=\begin{bmatrix} a_{[0][0]} & a_{[0][1]} & a_{[0][2]} \\ a_{[1][0]} & a_{[1][1]} & a_{[1][2]} \end{bmatrix}$$

$a_{[0][0]}$	$a_{[0][1]}$	$a_{[0][2]}$	$a_{[1][0]}$	$a_{[1][1]}$	$a_{[1][2]}$
第1行			第2行		

图 3.18　二维数组按行优先顺序存放

当以列优先方式存储时，得出存储结构关系式为

$$\mathrm{LOC}(a_{i,j})=\mathrm{LOC}(a_{0,0})+[j\times(h_1+1)+i]\times L$$

例如，对于数组 $\boldsymbol{A}_{[2][3]}$，它按列优先方式在内存中的存储形式如图 3.19 所示。

$$A_{[2][3]}=\begin{bmatrix} a_{[0][0]} & a_{[0][1]} & a_{[0][2]} \\ a_{[1][0]} & a_{[1][1]} & a_{[1][2]} \end{bmatrix}$$

$a_{[0][0]}$	$a_{[1][0]}$	$a_{[0][1]}$	$a_{[1][1]}$	$a_{[0][2]}$	$a_{[1][2]}$
第1列		第2列		第3列	

图 3.19　二维数组按列优先顺序存放

王道计算机教育 www.cskaoyan.com

3.4.3　特殊矩阵的压缩存储

压缩存储：指为多个值相同的元素只分配一个存储空间，对零元素不分配空间。

特殊矩阵：指具有许多相同矩阵元素或零元素，并且这些相同矩阵元素或零元素的分布有一定规律性的矩阵。常见的特殊矩阵有对称矩阵、上（下）三角矩阵、对角矩阵等。

特殊矩阵的压缩存储方法：找出特殊矩阵中值相同的矩阵元素的分布规律，把那些呈现规律性分布的、值相同的多个矩阵元素压缩存储到一个存储空间中。

1．对称矩阵

命题追踪 ▶▶ 对称矩阵压缩存储的下标对应关系（2018、2020）

若对一个 n 阶矩阵 $\boldsymbol{A}$ 中的任意一个元素 $a_{i,j}$ 都有 $a_{i,j}=a_{j,i}$（$1\leqslant i,j\leqslant n$），则称其为对称矩阵。其中的元素可以划分为 3 个部分，即上三角区、主对角线和下三角区，如图 3.20 所示。

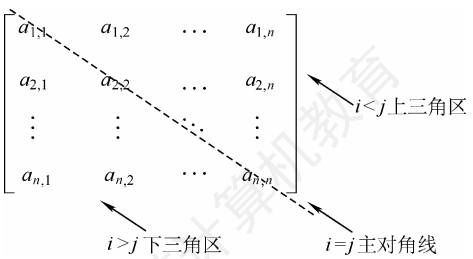


图 3.20　n 阶矩阵的划分

配套课程公众号[考机研]
永久联系微信:4550060

对于 n 阶对称矩阵，上三角区的所有元素和下三角区的对应元素相同，若仍采用二维数组存放，则会浪费几乎一半的空间，为此将 n 阶对称矩阵 $\boldsymbol{A}$ 存放在一维数组 B[n(n+1)/2]中，即元素 $a_{i,j}$ 存放在 b_k 中。比如只存放下三角部分（含主对角线）的元素。

在数组 B 中，位于元素 $a_{i,j}$（$i \geqslant j$）前面的元素个数为

第 1 行：1 个元素（$a_{1,1}$）。

第 2 行：2 个元素（$a_{2,1}$, $a_{2,2}$）。

……

第 $i-1$ 行：$i-1$ 个元素（$a_{i-1,1}, a_{i-1,2}, \cdots, a_{i-1,i-1}$）。

第 i 行：$j-1$ 个元素（$a_{i,1}, a_{i,2}, \cdots, a_{i,j-1}$）。

因此，元素 $a_{i,j}$ 在数组 B 中的下标 $k=1+2+\cdots+(i-1)+j-1=i(i-1)/2+j-1$（数组下标从 0 开始）。因此，元素下标之间的对应关系如下：

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j-1, & i \geqslant j \text{（下三角区和主对角线元素）} \\ \dfrac{j(j-1)}{2}+i-1, & i<j \text{（上三角区元素} a_{i,j}=a_{j,i}\text{）}\end{cases}$$

当数组下标从 1 开始时，可以采用同样的推导方法，请读者自行思考。

注 意

二维数组 A[n][n]和 A[0…n-1][0…n-1]的写法是等价的。若数组写成 A[1…n][1…n]，则表示指定了下标是从 1 开始的。二维数组元素写为 a[i][j]，注意数组元素下标 i 和 j 通常是从 0 开始的。矩阵元素通常写为 $a_{i,j}$，行号 i 和列号 j 通常是从 1 开始的。

王道计算机教育 www.cskaoyan.com

2．三角矩阵

下三角矩阵［见图 3.22(a)］中，上三角区的所有元素均为同一常量。其存储思想与对称矩阵类似，不同之处在于存储完下三角区和主对角线上的元素之后，紧接着存储对角线上方的常量一次，所以可以将 n 阶下三角矩阵 $\boldsymbol{A}$ 压缩存储在 B[n(n+1)/2+1]中。

元素下标之间的对应关系为

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j-1, & i \geqslant j \text{（下三角区和主对角线元素）} \\ \dfrac{n(n+1)}{2}, & i<j \text{（上三角区元素）}\end{cases}$$

下三角矩阵在内存中的压缩存储形式如图 3.21 所示。

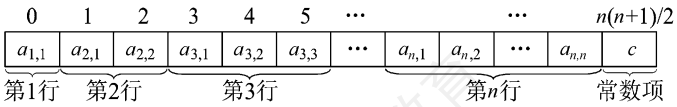


图 3.21　下三角矩阵的压缩存储

命题追踪 ▶▶ 上三角矩阵采用行优先存储的应用（2011）

上三角矩阵［见图 3.22(b)］中，下三角区的所有元素均为同一常量。只需存储主对角线、上三角区上的元素和下三角区的常量一次，可将其压缩存储在 B[n(n+1)/2+1]中。

配套课程公众号[考机研]
永久联系微信:4550060

$$\begin{bmatrix} a_{1,1} & & & \\ a_{2,1} & a_{2,2} & & \\ \vdots & \vdots & \ddots & \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix} \qquad \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ & a_{2,2} & \cdots & a_{2,n} \\ & & \ddots & \vdots \\ & & & a_{n,n} \end{bmatrix}$$

(a) 下三角矩阵　　(b) 上三角矩阵

图 3.22　三角矩阵

在数组 B 中，位于元素 $a_{i,j}$（$i \leqslant j$）前面的元素个数为

第 1 行：n 个元素

第 2 行：$n-1$ 个元素

……

第 $i-1$ 行：$n-i+2$ 个元素

第 i 行：$j-i$ 个元素

因此，元素 $a_{i,j}$ 在数组 B 中的下标

$$k = n+(n-1)+\cdots+(n-i+2)+(j-i+1)-1 = (i-1)(2n-i+2)/2+(j-i)$$

因此，元素下标之间的对应关系如下：

$$k = \begin{cases} \dfrac{(i-1)(2n-i+2)}{2}+(j-i), & i \leqslant j \text{（上三角区和主对角线元素）} \\ \dfrac{n(n+1)}{2}, & i > j \text{（下三角区元素）} \end{cases}$$

上三角矩阵在内存中的压缩存储形式如图 3.23 所示。

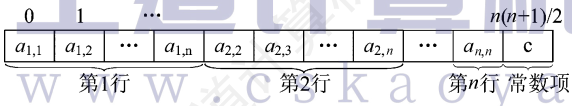


图 3.23　上三角矩阵的压缩存储

以上推导均假设数组的下标从 0 开始，若题设有具体要求，则应该灵活应对。

3．三对角矩阵

对角矩阵也称带状矩阵。对 n 阶矩阵 $\boldsymbol{A}$ 中的任意一个元素 $a_{i,j}$，当 $|i-j|>1$ 时，若有 $a_{i,j}=0$（$1 \leqslant i,j \leqslant n$），则称为三对角矩阵，如图 3.24 所示。在三对角矩阵中，所有非零元素都集中在以主对角线为中心的 3 条对角线的区域，其他区域的元素都为零。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & 0 & \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

图 3.24　三对角矩阵 $\boldsymbol{A}$

三对角矩阵 $\boldsymbol{A}$ 也可以采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一维数组 B 中，且 $a_{1,1}$ 存放于 B[0]中，其存储形式如图 3.25 所示。

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	…	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$

图 3.25　三对角矩阵的压缩存储

配套课程公众号[考机研]
永久联系微信:4550060

命题追踪 ▶▶ 三对角矩阵压缩存储的下标对应关系（2016）

由此可以计算矩阵 A 中 3 条对角线上的元素 $a_{i,j}$（$1 \leqslant i, j \leqslant n, |i-j| \leqslant 1$）在一维数组 B 中存放的下标为 $k=2i+j-3$。

反之，若已知三对角矩阵中的某个元素 $a_{i,j}$ 存放在一维数组 B 的第 k 个位置，则有 $i=\lfloor (k+1)/3+1 \rfloor$，$j=k-2i+3$。例如，当 $k=0$ 时，$i=\lfloor (0+1)/3+1 \rfloor=1$，$j=0-2\times1+3=1$，存放的是 $a_{1,1}$；当 $k=2$ 时，$i=\lfloor (2+1)/3+1 \rfloor=2$，$j=2-2\times2+3=1$，存放的是 $a_{2,1}$；当 $k=4$ 时，$i=\lfloor (4+1)/3+1 \rfloor=2$，$j=4-2\times2+3=3$，存放的是 $a_{2,3}$。

3.4.4　稀疏矩阵

矩阵中非零元素的个数 t，相对矩阵元素的个数 s 来说非常少，即 $s \gg t$ 的矩阵称为稀疏矩阵。例如，一个矩阵的阶为 100×100，该矩阵中只有少于 100 个非零元素。

命题追踪 ▶▶ 存储稀疏矩阵需要保存的信息（2023）

若采用常规的方法存储稀疏矩阵，则相当浪费存储空间，因此仅存储非零元素。但通常非零元素的分布没有规律，所以仅存储非零元素的值是不够的，还要存储它所在的行和列。因此，将非零元素及其相应的行和列构成一个三元组（行标 i，列标 j，值 $a_{i,j}$），如图 3.26 所示。然后按照某种规律存储这些三元组线性表。稀疏矩阵压缩存储后便失去了随机存取特性。

$$M=\begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 9 & 0 & 0 \\ 0 & 23 & 0 & 0 \end{bmatrix}$$

对应的三元组

i	j	$a_{i,j}$
0	0	4
1	2	6
2	1	9
3	1	23

图 3.26　稀疏矩阵及其对应的三元组

命题追踪 ▶▶ 适合稀疏矩阵压缩存储的存储结构（2017）

稀疏矩阵的三元组表既可以采用数组存储，又可以采用十字链表存储（见 6.2 节）。当存储稀疏矩阵时，不仅要保存三元组表，而且要保存稀疏矩阵的行数、列数和非零元素的个数。

3.4.5　本节试题精选

单项选择题

01. 对特殊矩阵采用压缩存储的主要目的是（　）。

A. 表达变得简单　　B. 对矩阵元素的存取变得简单

C. 去掉矩阵中的多余元素　　D. 减少不必要的存储空间

02. 对 n 阶对称矩阵压缩存储时，需要表长为（　）的顺序表。

A. $n/2$　　B. $n\times n/2$　　C. $n(n+1)/2$　　D. $n(n-1)/2$

03. 有一个 $n\times n$ 的对称矩阵 A，将其下三角部分按行存放在一维数组 B 中，而 A[0][0]存放于 B[0]中，则元素 A[i][i]存放于 B 中的（　）处。

A. $(i+3)i/2$　　B. $(i+1)i/2$　　C. $(2n-i+1)i/2$　　D. $(2n-i-1)i/2$

04. 在二维数组 A 中，假设每个数组元素的长度为 3 个存储单元，行下标 i 为 0～8，列下标 j 为 0～9，从首地址 SA 开始连续存放。在这种情况下，元素 A[8][5]的起始地址为（　）。

A. SA+141　　B. SA+144　　C. SA+222　　D. SA+255

05. 二维数组 A 按行优先存储，其中每个元素占 1 个存储单元。若 A[1][1]的存储地址为

配套课程公众号[考机研]
永久联系微信:4550060

420，A[3][3]的存储地址为 446，则 A[5][5]的存储地址为（　）。

A. 472　　B. 471　　C. 458　　D. 457

06. 将三对角矩阵即数组 A[1…100][1…100]按行优先存入一维数组 B[1…298]中，数组 A 中元素 A[66][65]在数组 B 中的位置 k 为（　）。

A. 198　　B. 195　　C. 197　　D. 196

07. 若将 n 阶上三角矩阵 $\boldsymbol{A}$ 按列优先级压缩存放在一维数组 B[1…n(n+1)/2+1]中，则存放到 B[k]中的非零元素 $a_{i,j}$（$1\leqslant i,j\leqslant n$）的下标 i、j 与 k 的对应关系是（　）。

A. $i(i+1)/2+j$　　B. $i(i-1)/2+j-1$　　C. $j(j-1)/2+i$　　D. $j(j-1)/2+i-1$

08. 若将 n 阶下三角矩阵 $\boldsymbol{A}$ 按列优先顺序压缩存放在一维数组 B[1…n(n+1)/2+1]中，则存放到 B[k]中的非零元素 $a_{i,j}$（$1\leqslant i,j\leqslant n$）的下标 i,j 与 k 的对应关系是（　）。

A. $(j-1)(2n-j+1)/2+i-j$　　B. $(j-1)(2n-j+2)/2+i-j+1$

C. $(j-1)(2n-j+2)/2+i-j$　　D. $(j-1)(2n-j+1)/2+i-j-1$

09. 稀疏矩阵采用压缩存储后的缺点主要是（　）。

A. 无法判断矩阵的行列数　　B. 丧失随机存取的特性

C. 无法由行、列值查找某个矩阵元素　　D. 使矩阵元素之间的逻辑关系更复杂

10. 下列关于矩阵的说法中，正确的是（　）。

I. 在 n（$n>3$）阶三对角矩阵中，每行都有 3 个非零元

II. 稀疏矩阵的特点是矩阵中的元素较少

A. 仅 I　　B. 仅 II　　C. I 和 II　　D. 无正确项

11. 【2016 统考真题】有一个 100 阶的三对角矩阵 $\boldsymbol{M}$，其元素 $m_{i,j}$（$1\leqslant i,j\leqslant 100$）按行优先依次压缩存入下标从 0 开始的一维数组 N 中。元素 $m_{30,30}$ 在 N 中的下标是（　）。

A. 86　　B. 87　　C. 88　　D. 89

12. 【2017 统考真题】适用于压缩存储稀疏矩阵的两种存储结构是（　）。

A. 三元组表和十字链表　　B. 三元组表和邻接矩阵

C. 十字链表和二叉链表　　D. 邻接矩阵和十字链表

13. 【2018 统考真题】设有一个 12×12 阶对称矩阵 $\boldsymbol{M}$，将其上三角部分的元素 $m_{i,j}$（$1\leqslant i\leqslant j\leqslant 12$）按行优先存入 C 语言的一维数组 N 中，元素 $m_{6,6}$ 在 N 中的下标是（　）。

A. 50　　B. 51　　C. 55　　D. 66

14. 【2020 统考真题】将一个 10×10 阶对称矩阵 $\boldsymbol{M}$ 的上三角部分的元素 $m_{i,j}$（$1\leqslant i\leqslant j\leqslant 10$）按列优先存入 C 语言的一维数组 N 中，元素 $m_{7,2}$ 在 N 中的下标是（　）。

A. 15　　B. 16　　C. 22　　D. 23

15. 【2021 统考真题】二维数组 A 按行优先方式存储，每个元素占用 1 个存储单元。若元素 A[0][0]的存储地址是 100，A[3][3]的存储地址是 220，则元素 A[5][5]的存储地址是（　）。

A. 295　　B. 300　　C. 301　　D. 306

16. 【2023 统考真题】若采用三元组表存储结构存储稀疏矩阵 $\boldsymbol{M}$，则除三元组表外，下列数据中还需要保存的是（　）。

I. $\boldsymbol{M}$ 的行数　　II. $\boldsymbol{M}$ 中包含非零元素的行数

III. $\boldsymbol{M}$ 的列数　　IV. $\boldsymbol{M}$ 中包含非零元素的列数

A. 仅 I、III　　B. 仅 I、IV　　C. 仅 II、IV　　D. I、II、III、IV

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

3.4.6 答案与解析

单项选择题

01. D

特殊矩阵中含有很多相同元素或零元素，所以可采用压缩存储，以节省存储空间。

02. C

只需存储其上三角或下三角部分（含对角线），元素个数为 $n+(n-1)+\cdots+1=n(n+1)/2$。

03. A

此题要注意3个细节：矩阵的最小下标为0；数组下标也是从0开始的；矩阵按行优先存在数组中。注意到此三点，答案不难得到为选项A。此外，本类题建议采用特殊值代入法求解，例如，A[1][1]对应的下标应为2，代入后只有选项A满足条件。

技 巧

对于特殊三角矩阵压缩存储的题，心中应有“平移”搬动的思想，并结合草图，这样会比较形象，在计算时再注意矩阵和数组的起始下标，就不容易出错。

04. D

二维数组计算地址（按行优先顺序）的公式为

$$\mathrm{LOC}(i,j)=\mathrm{LOC}(0,0)+(i\times m+j)\times L$$

其中，LOC(0, 0) = SA，是数组存放的首地址；$L=3$ 是每个数组元素的长度；$m=9-0+1=10$ 是数组的列数。因此有 $\mathrm{LOC}(8,5)=\mathrm{SA}+(8\times10+5)\times3=\mathrm{SA}+255$，所以选择选项D。

05. A

本题未直接给出数组A的行数和列数，因此需要根据题目中的信息来推理。因为该二维数组按行优先存储，且A[3][3]的存储地址为446，所以A[3][1]的存储地址为444，又A[1][1]的存储地址为420，显然A[1][1]和A[3][1]正好相差2行，所以该矩阵的列数为12。而A[5][3]和A[3][3]正好相差2行，A[5][5]和A[5][3]又相差2个元素，所以A[5][5]的存储地址是446+24+2=472。

06. B

对于三对角矩阵，将A[1…n][1…n]压缩至B[1…3n-2]时，$a_{i,j}$ 与 b_k 的对应关系为 $k=2i+j-2$。则A中的元素A[66][65]在数组B中的位置 k 为 $2\times66+65-2=195$。

07. C

按列优先存储，所以元素 $a_{i,j}$ 前面有 $j-1$ 列，共有 $1+2+3+\cdots+j-1=j(j-1)/2$ 个元素，元素 $a_{i,j}$ 在第 j 列上是第 i 个元素，数组 B 的下标是从1开始，因此 $k=j(j-1)/2+i$。

08. B

按列优先存储，所以元素 $a_{i,j}$ 之前有 $j-1$ 列，共有 $n+(n-1)+\cdots+(n-j+2)=(j-1)(2n-j+2)/2$ 个元素，元素 $a_{i,j}$ 是第 j 列上第 $i-j+1$ 个元素，数组B的下标从1开始，$k=(j-1)(2n-j+2)/2+i-j+1$。

09. B

稀疏矩阵通常采用三元组来压缩存储，存储矩阵元素的行列下标和相应的值，因此不能根据矩阵元素的行列下标快速定位矩阵元素，失去了随机存取的特性。

10. D

在三对角矩阵中，第1行和最后1行只有2个非零元，其余各行均有3个非零元。稀疏矩阵的特点是矩阵中非零元的个数较少。

王道计算机教育 www.wangdaojiaoyu.com
配套课程公众号[考机研]
永久联系微信:4550060

11．B

三对角矩阵如下所示。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & 0 & \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

采用压缩存储，将 3 条对角线上的元素按行优先方式存放在一维数组 B 中，且 $a_{1,1}$ 存放于 B[0] 中，其存储形式如下所示：

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	…	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$

可以计算矩阵 $\boldsymbol{A}$ 中 3 条对角线上的元素 $a_{i,j}$（$1 \leqslant i,j \leqslant n, |i-j| \leqslant 1$）在一维数组 B 中存放的下标为 $k=2i+j-3$，公式很难记忆，我们通常采用解法 2。

解法 1：针对该题，仅需将数字逐一代入公式：$k=2\times30+30-3=87$，结果为 87。

解法 2：观察上图的三对角矩阵不难发现，第一行有两个元素，剩下的在元素 $m_{30,30}$ 所在行之前的 28 行（注意下标 $1 \leqslant i,j \leqslant 100$）中，每行都有 3 个元素，而 $m_{30,30}$ 之前仅有一个元素 $m_{30,29}$，不难发现元素 $m_{30,30}$ 在数组 N 中的下标是 $2+28\times3+2-1=87$。

注　意

矩阵和数组的下标从 0 或 1 开始（如矩阵可能从 $a_{0,0}$ 或 $a_{1,1}$ 开始，数组可能从 B[0] 或 B[1] 开始），这时就需要适时调整计算方法（方法无非是多计算 1 或少计算 1 的问题）。

12．A

三元组表的结点存储了行（row）、列（col）、值（value）三种信息，是主要用来存储稀疏矩阵的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达 $O(n^2)$，不适合于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法，可用于表示树或森林。

13．A

在 C 语言中，数组 N 的下标从 0 开始。第一个元素 $m_{1,1}$ 对应存入 n_0，矩阵 $\boldsymbol{M}$ 的第一行有 12 个元素，第二行有 11 个，第三行有 10 个，第四行有 9 个，第五行有 8 个，所以 $m_{6,6}$ 是第 $12+11+10+9+8+1=51$ 个元素，下标应为 50。

14．C

上三角矩阵按列优先存储，先存储只有 1 个元素的第一列，再存储有 2 个元素的第二列，以此类推。$m_{7,2}$ 位于左下角，对应右上角的元素为 $m_{2,7}$，在 $m_{2,7}$ 之前存有

第 1 列：1

第 2 列：2

……

第 6 列：6

第 7 列：1

前面共存储有 $1+2+3+4+5+6+1=22$ 个元素（数组下标范围为 0～21），注意数组下标从 0 开始，所以 $m_{2,7}$ 在数组 N 中的下标为 22，即 $m_{7,2}$ 在数组 N 中的下标为 22。

配套课程公众号[考机研]
永久联系微信:4550060

15. B

二维数组 A 按行优先存储，每个元素占用 1 个存储单元，由 A[0][0]和 A[3][3]的存储地址可知 A[3][3]是二维数组 A 中的第 121 个元素，假设二维数组 A 的每行有 n 个元素，则 $n\times3+4=121$，求得 $n=39$，所以元素 A[5][5]的存储地址为 $100+39\times5+6-1=300$。

16. A

用三元组表存储结构存储稀疏矩阵 $\boldsymbol{M}$ 时，每个非零元素都由三元组（行标、列标、关键字值）组成。但是，仅通过三元组表中的元素无法判断稀疏矩阵 $\boldsymbol{M}$ 的大小，因此还要保存 $\boldsymbol{M}$ 的行数和列数。此外，还可以保存 $\boldsymbol{M}$ 的非零元素个数。例如，如下两个稀疏矩阵的三元组表是相同的，若不保存行数和列数，则无法判断两个稀疏矩阵的大小。

0	2	0	0
0	0	5	0
0	0	0	0
0	0	0	0

0	2	0
0	0	5
0	0	0

归纳总结

本章所讲的几种数据结构类型是线性表的应用和推广，在考试中主要以选择题形式进行考查，但栈和队列也仍然有可能出现在算法设计题中。很多读者看到课本上有好多个函数时很恐惧，若考到了栈或队列的大题，难道要把每个操作的函数都写出来吗？

其实，在考试中，栈或队列都是作为一个工具来解决其他问题的，我们可以把栈或队列的声明和操作写得很简单，而不必分函数写出。以顺序栈的操作为例：

（1）声明一个栈并初始化：

```
Elemtype stack[maxSize]; int top=-1; //两句话连声明带初始化都有了
```

（2）元素入栈：

```
stack[++top]=x;      //仅一句话即实现入栈操作
```

（3）元素 x 出栈:

```
X=stack[top--];      //单目运算符在变量之前表示“先运算后使用”，之后则相反
```

对于链式栈，同样只需定义一个结构体，然后从讲解中摘取必要的语句组合在自己的函数代码中即可。另外，在考研真题中，链式栈出现的概率要比顺序栈低得多，因此大家应该有所侧重，多训练与顺序栈相关的题目。

思维拓展

设计一个栈，使它可以在 $O(1)$的时间复杂度内实现 Push、Pop 和 min 操作。所谓 min 操作，是指得到栈中最小的元素。

提　示

使用双栈，两个栈是同步关系。主栈是普通栈，用来实现栈的基本操作 Push 和 Pop；辅助栈用来记录同步的最小值 min，例如元素 x 入栈，则辅助栈 stack_min[top++]= (x<min)?x:min; 即在每次 Push 中，都将当前最小元素放到 stack_min 的栈顶。在主栈中 Pop 最小元素 y 时，stack_min 栈中相同位置的最小元素 y 也会随着 top--而出栈。因此 stack_min 的栈顶元素必然是 y 之前入栈的最小元素。本题是典型的以空间换时间的算法。

配套课程公众号[考机研]
永久联系微信:4550060